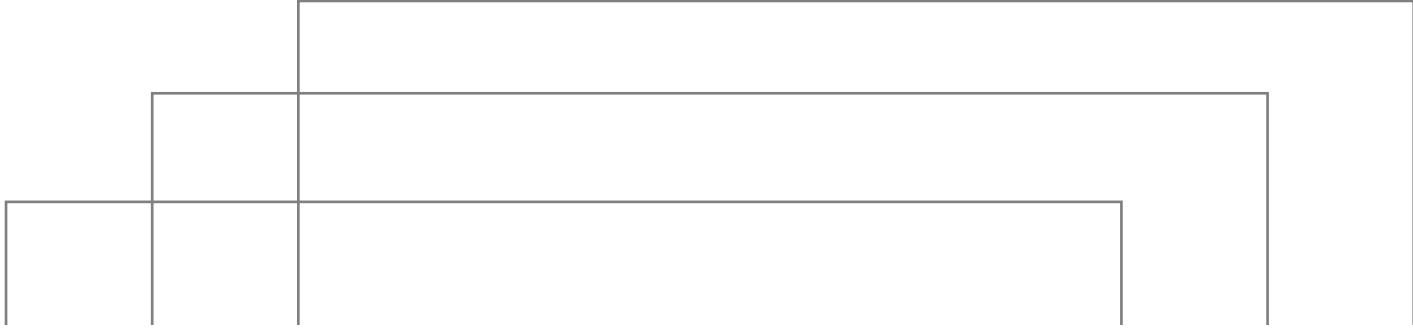
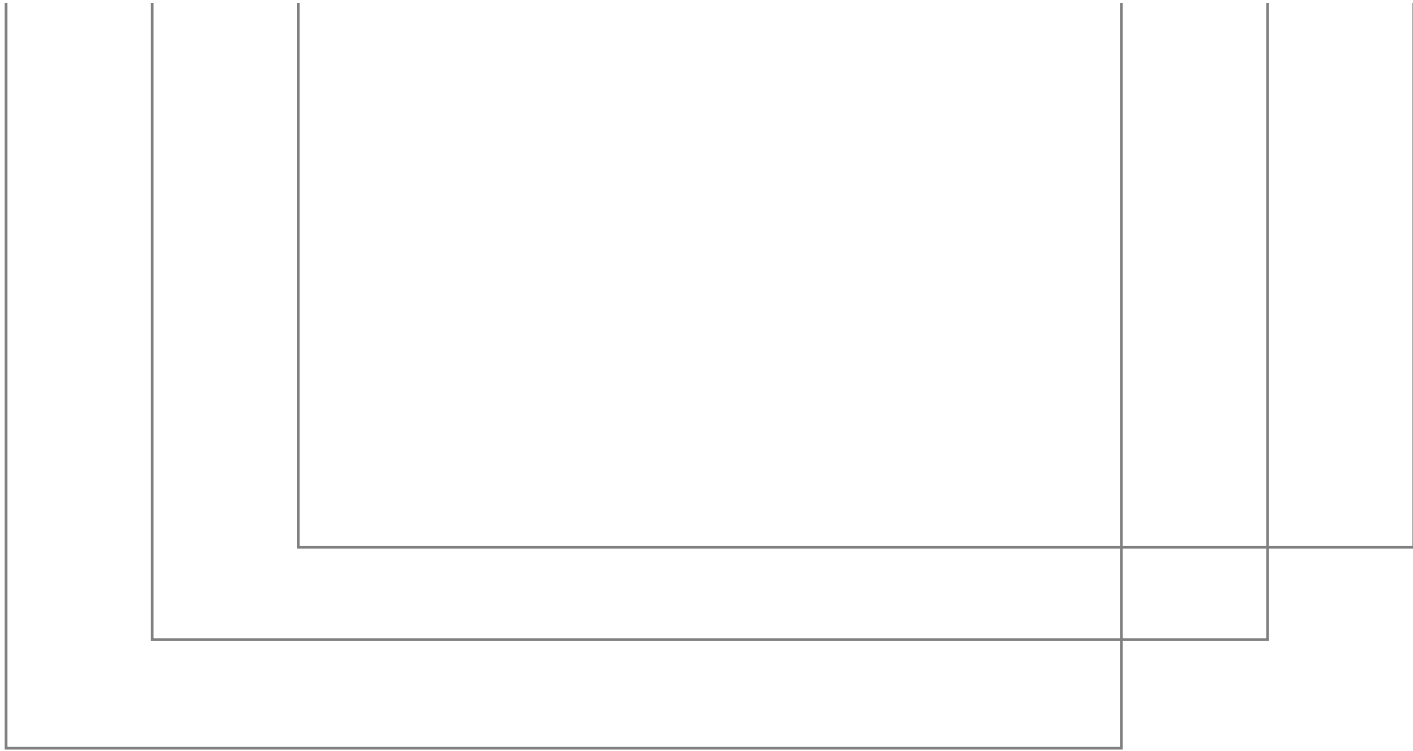




Multiple Pages App, Navigation and Routing



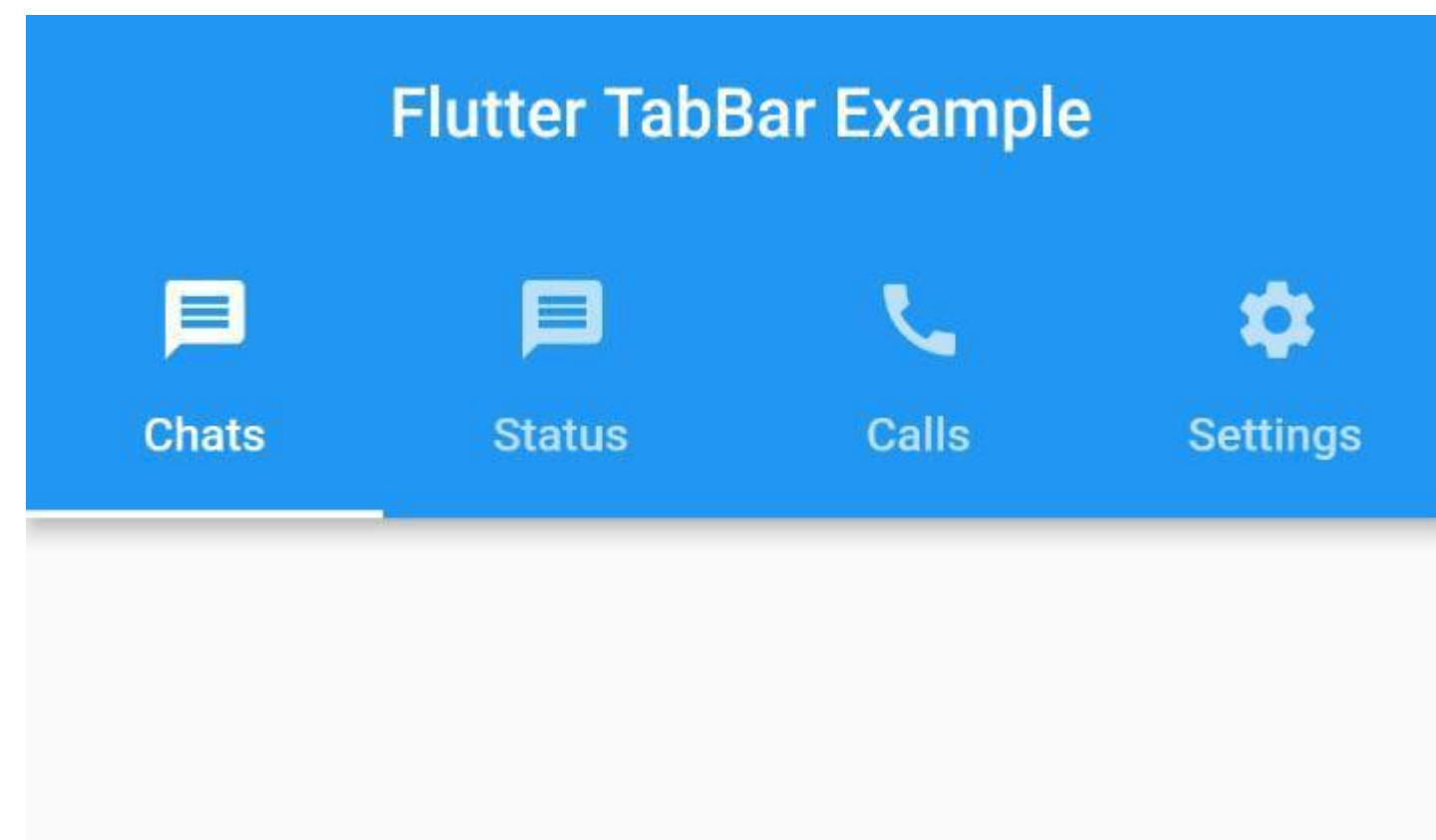
By Prasertsak U., Ph.D

Outlines

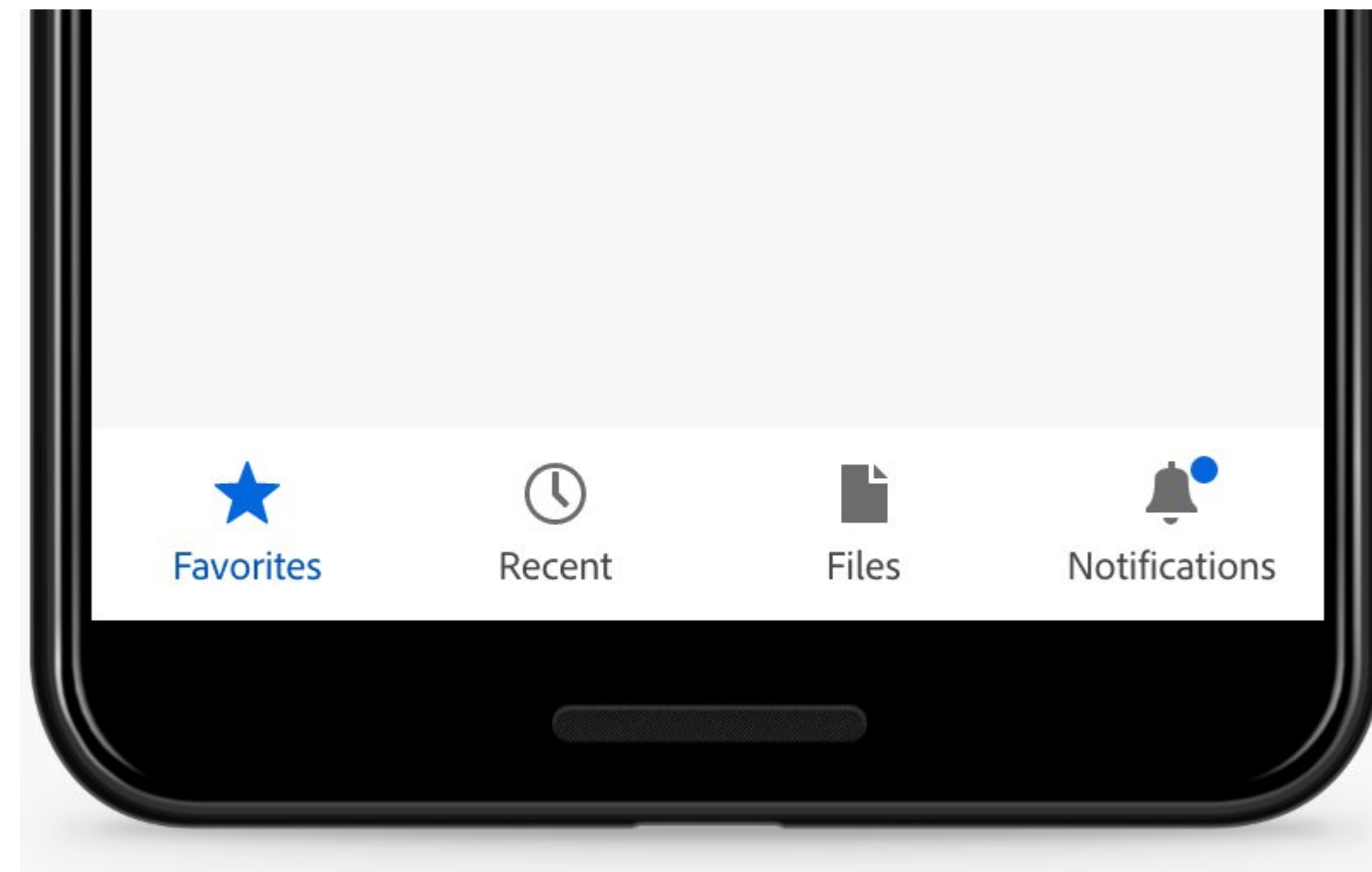
- Using TabBar widget
- Navigate to a new page
- Send data to a new page
- return data from page

Multiple Page Widgets – Style

Tab Bar



Bottom Navigation Bar



Using **TabBar** widget

- Working with tabs is a common pattern in apps.
- สรุปขั้นตอนในการทำงาน Tab
 - 1) Create *TabController* – using *DefaultTabController*
 - 2) Create the Tabs
 - 3) Create specific content of each tab

1. Create a *TabController*

- ใช้ widget ชื่อว่า *DefaultTabController* ซึ่งจะต้องระบุจำนวน Tab ไว้ในตัวแปร *length* และกำหนดรายละเอียดของ tabs ต่างๆ ไว้ในส่วนของ child ดังตัวอย่าง

```
return MaterialApp(  
  home: DefaultTabController(length: 3, child: Scaffold()),  
);
```

2. Create the *Tabs*

- ใช้ widget ชื่อว่า **TabBar** ซึ่งจะต้องระบุรายละเอียดของ Tabs ต่างๆ ไว้ในส่วนของ widget ชื่อว่า **Tab** โดยจำนวน Tab จะต้องสอดคล้องกับค่าของ **length** ที่กำหนดไว้ก่อนหน้านี้ ดังตัวอย่าง กำหนด length ไว้ 3 ดังนั้นจำนวนสมาชิกของ Tabs จึงมี 3 Tab() เช่นกัน

```
return MaterialApp(  
  home: DefaultTabController(  
    length: 3,  
    child: Scaffold(  
      appBar: AppBar(  
        bottom: const TabBar(  
          tabs: [  
            Tab(icon: Icon(Icons.directions_car)),  
            Tab(icon: Icon(Icons.directions_transit)),  
            Tab(icon: Icon(Icons.directions_bike)),  
          ],  
        ),  
      ),  
    ),  
  ),  
);
```

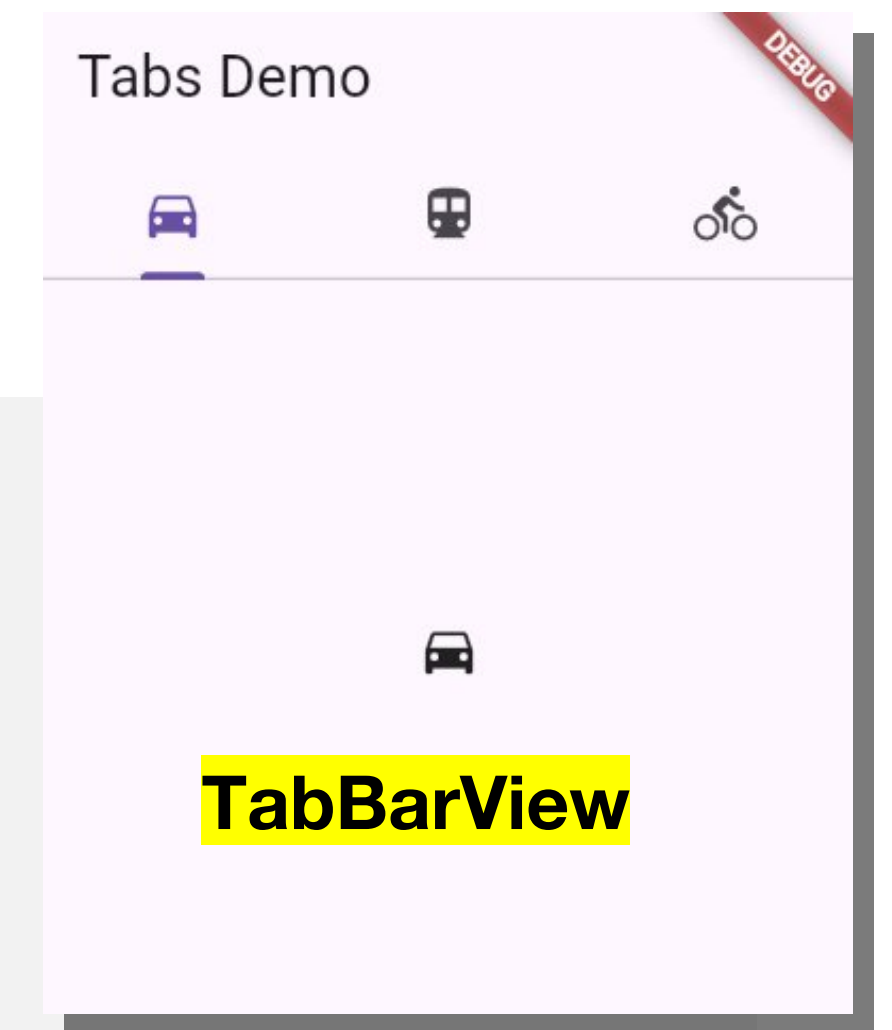

3. Create specific content of each tab

- สร้าง content ที่จะแสดงเมื่อคลิกหรือแตะที่ tab ใด ๆ โดยจะกำหนด content ต่าง ๆ ไว้ใน widget ชื่อว่า **TabBarView** ทั้งนี้จำนวน content จะต้องสอดคล้องกับจำนวน Tab ที่ระบุไว้ก่อนหน้านี้เช่นกัน ดังตัวอย่าง เนื้อหาที่จะปรากฏในแต่ละ Tab คือภาพไอคอน

```
body: const TabBarView(  
  children: [  
    Icon(Icons.directions_car),  
    Icon(Icons.directions_transit),  
    Icon(Icons.directions_bike),  
  ],  
),
```

ตัวอย่าง #1

TabBar



```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const TabBarDemo());  
}
```

```
class TabBarDemo extends StatelessWidget {  
  const TabBarDemo({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(  
      );
```

```
  }  
}
```

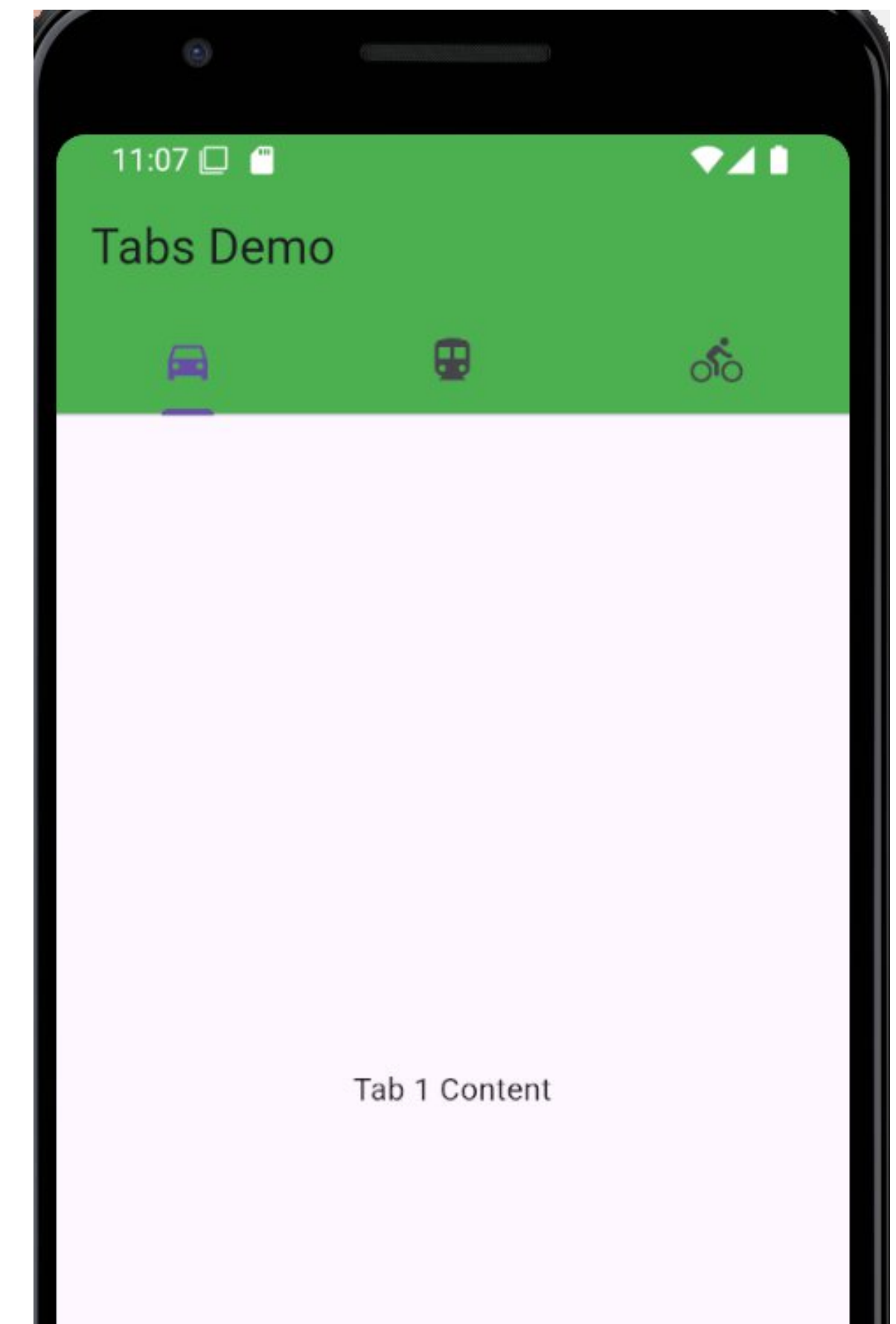
```
return MaterialApp(  
  home: DefaultTabController(  
    length: 3,  
    child: Scaffold(  
      appBar: AppBar(  
        bottom: const TabBar(  
          tabs: [  
            Tab(icon: Icon(Icons.directions_car)),  
            Tab(icon: Icon(Icons.directions_transit)),  
            Tab(icon: Icon(Icons.directions_bike)),  
          ],  
        ),  
        title: const Text('Tabs Demo'),  
      ),  
      body: const TabBarView(  
        children: [  
          Icon(Icons.directions_car),  
          Icon(Icons.directions_transit),  
          Icon(Icons.directions_bike),  
        ],  
      ),  
    ),  
  ),  
);
```


ตัวอย่าง #2

- เราสามารถสร้างส่วนของเนื้อหาแยกเป็น method และ return เป็นก้อน widget เพื่อนำไปใช้ในส่วนของ TabBarView ได้ เช่น

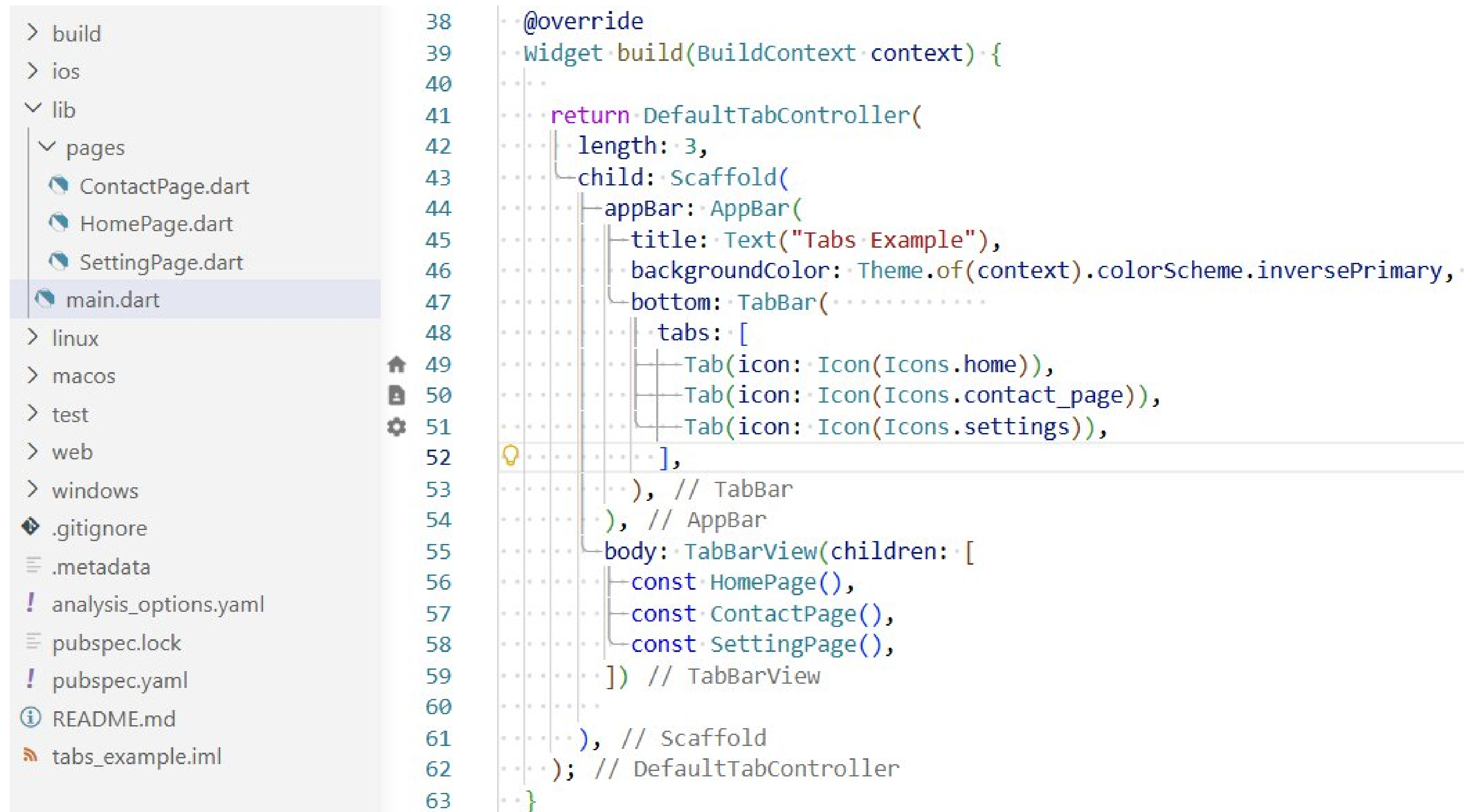
```
body: TabBarView(  
  children: [  
    tab1(),  
    tab2(),  
    tab3(),  
  ],  
),
```

```
Widget tab1() {  
  return const Center(  
    child: Text('Tab 1 Content'),  
  );  
}  
Widget tab2() {  
  return const Center(  
    child: Text('Tab 2 Content'),  
  );  
}  
Widget tab3() {  
  return const Center(  
    child: Text('Tab 3 Content'),  
  );  
}
```

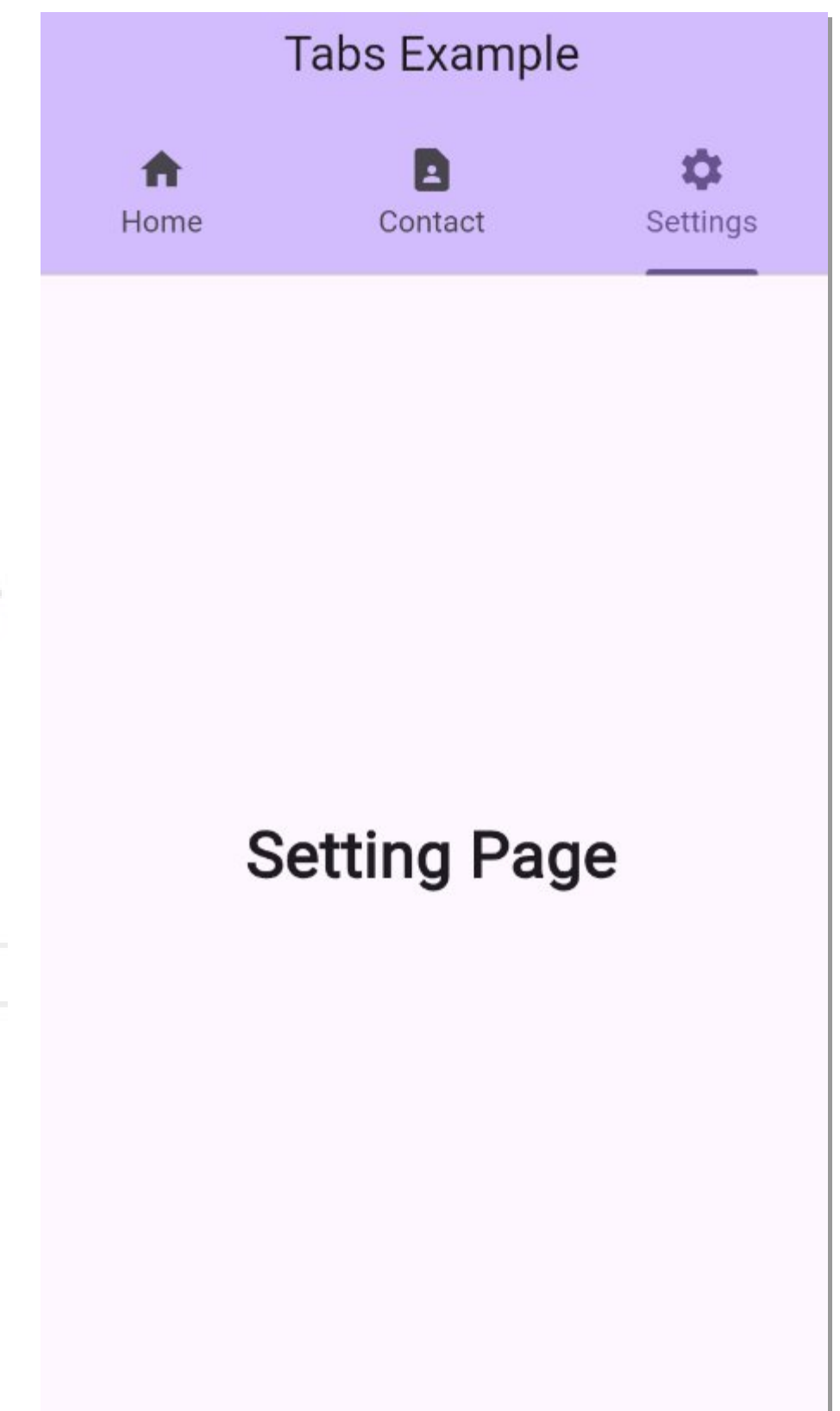


ตัวอย่าง #3

- สามารถสร้างเนื้อหาของแต่ละ tab แยกเป็น class โดยสร้างเป็นไฟล์ และนำมาใช้ในส่วนของ `TabBarView()` ได้เช่นกัน ซึ่งจะทำให้โค้ดเป็นสัดส่วน และอ่านง่ายขึ้น



```
38  @override
39  Widget build(BuildContext context) {
40    ...
41    return DefaultTabController(
42      length: 3,
43      child: Scaffold(
44        appBar: AppBar(
45          title: Text("Tabs Example"),
46          backgroundColor: Theme.of(context).colorScheme.inversePrimary,
47          bottom: TabBar(
48            tabs: [
49              Tab(icon: Icon(Icons.home)),
50              Tab(icon: Icon(Icons.contact_page)),
51              Tab(icon: Icon(Icons.settings)),
52            ],
53          ), // TabBar
54        ), // AppBar
55        body: TabBarView(children: [
56          const HomePage(),
57          const ContactPage(),
58          const SettingPage(),
59        ]) // TabBarView
60      ), // Scaffold
61    ); // DefaultTabController
62  }
63 }
```



ตัวอย่าง #3 – รายละเอียดเนื้อหาภายในไฟล์ที่สร้างแยกเตรียมไว้

```
1  import 'package:flutter/material.dart';
2
3  class HomePage extends StatelessWidget {
4    const HomePage({super.key});
5
6    // This widget is the root of your application.
7    @override
8    Widget build(BuildContext context) {
9      return Container(
10        padding: EdgeInsets.all(20),
11        child: Center(
12          child: Text(
13            "Home Page",
14            style: TextStyle(fontSize: 30, fontWeight: FontWeight.bold),
15          ), // Text
16        ), // Center
17      ); // Container
18    }
19  }
```

ตัวอย่างหน้า Home

```
1  import 'package:flutter/material.dart';
2
3  class SettingPage extends StatelessWidget {
4    const SettingPage({super.key});
5
6    // This widget is the root of your application.
7    @override
8    Widget build(BuildContext context) {
9      return Container(
10        padding: EdgeInsets.all(20),
11        child: Center(
12          child: Text(
13            "Setting Page",
14            style: TextStyle(fontSize: 30, fontWeight: FontWeight.bold),
15          ), // Text
16        ), // Center
17      ); // Container
18    }
19  }
```

ตัวอย่างหน้า Setting

ข้อสรุปในการใช้ DefaultTabController

- จำนวน length จะต้องสอดคล้องกับจำนวนสมาชิกที่อยู่ใน **TabBar** และ **TabBarView**
- ลำดับของเนื้อหาภายใน TabBarView จะมีความหมายสอดคล้องกับลำดับของ Tabs

```
Widget build(BuildContext context) {  
  return DefaultTabController(  
    length: 3,  
    child: Scaffold(  
      appBar: AppBar(  
        backgroundColor: Colors.deepPurple,  
        bottom: const TabBar(  
          indicatorColor: Colors.amberAccent,  
          indicatorWeight: 4,  
          tabs: [  
            Tab(icon: Icon(Icons.home_outlined)),  
            Tab(icon: Icon(Icons.calculate_outlined)),  
            Tab(icon: Icon(Icons.info_outline)),  
          ],  
        ), // TabBar  
        title: const Text('Tabs Demo'),  
      ), // AppBar  
      body: const TabBarView(  
        children: [  
          Icon(Icons.home, size: 240),  
          Icon(Icons.calculate_outlined, size: 240),  
          Icon(Icons.info_outlined, size: 240),  
        ],  
      ),  
    ),  
  ),  
);
```


Bottom Navigation Bar

- เป็นการนำ tabs มาวางไว้บริเวณด้านล่างของหน้าจอ โดยสามารถกำหนดผ่าน bottomNavigationBar ของ Scaffold
- ค่าของ currentIndex จะเป็นตัวกำหนดว่า ผู้ใช้กดเลือกที่ tab ไດ จากนั้นจึงนำค่าของ index มาเป็นตัวชี้ไปยัง page ที่ต้องการแสดง

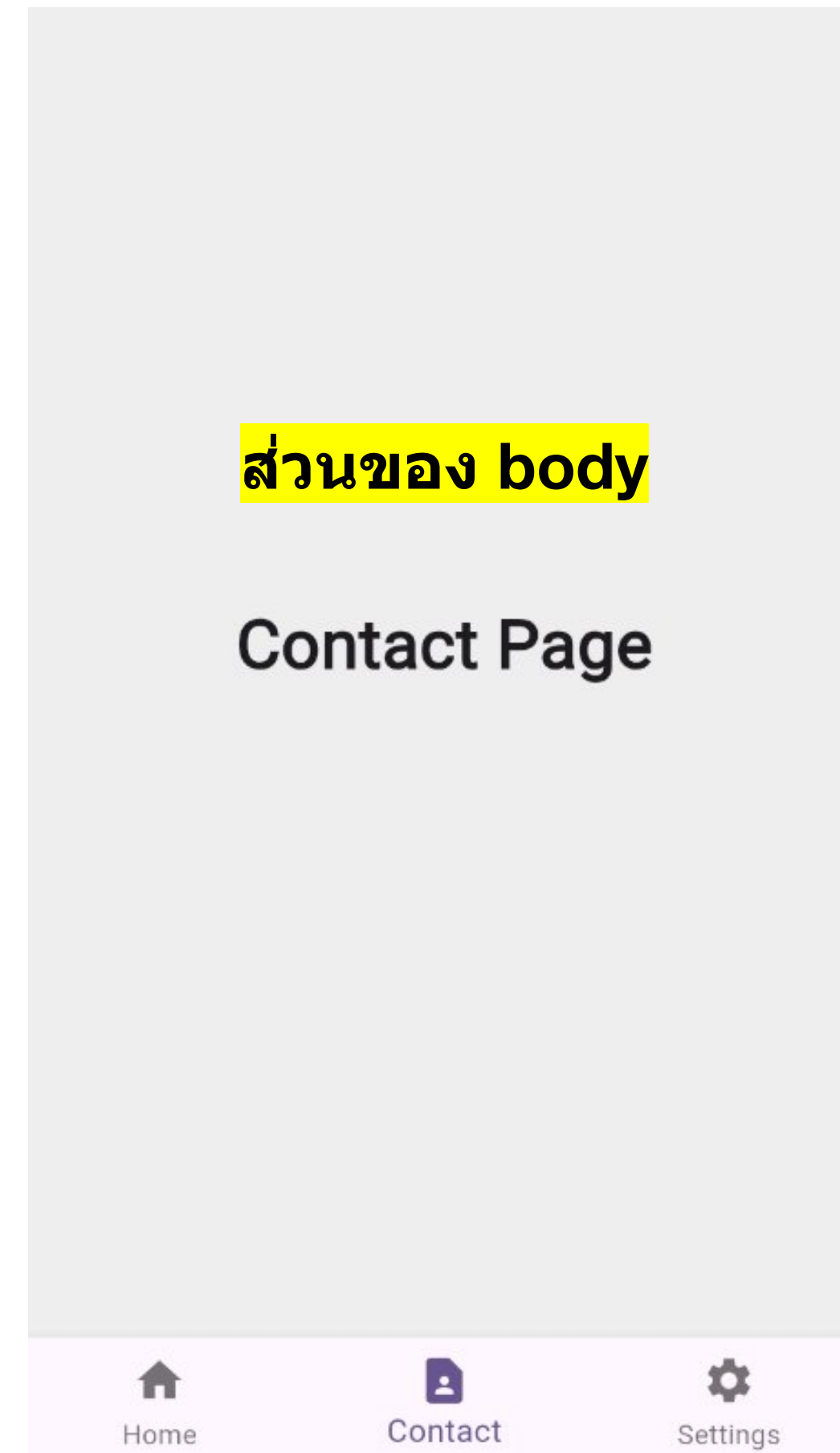
```
53 · @override
54 · Widget build(BuildContext context) {
55
56 ·   return Scaffold(
57 ·     bottomNavigationBar: BottomNavigationBar(
58 ·       currentIndex: _selectedIndex,
59 ·       onTap: (index) {
60 ·         setState(() {
61 ·           _selectedIndex = index;
62 ·         });
63 ·       },
64 ·       items: [
65 ·         BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
66 ·         BottomNavigationBarItem(icon: Icon(Icons.contact_page), label: "Contact"),
67 ·         BottomNavigationBarItem(icon: Icon(Icons.settings), label: "Settings"),
68 ·       ],
69 ·     ), // BottomNavigationBar
70 ·     body: _getSelectedPage(),
71 ·   ); // Scaffold
```

```
38 · int _selectedIndex = 0;
39
40 · Widget _getSelectedPage() {
41 ·   switch (_selectedIndex) {
42 ·     case 0:
43 ·       return const HomePage();
44 ·     case 1:
45 ·       return const ContactPage();
46 ·     case 2:
47 ·       return const SettingPage();
48 ·     default:
49 ·       return const HomePage();
50 ·   }
51 · }
```

Bottom Navigation Bar – ตัวอย่าง

```
36 class _MyHomePageState extends State<MyHomePage> {
37
38   int _selectedIndex = 0;
39
40   Widget _getSelectedPage() {
41     switch (_selectedIndex) {
42       case 0:
43         return const HomePage();
44       case 1:
45         return const ContactPage();
46       case 2:
47         return const SettingPage();
48       default:
49         return const HomePage();
50     }
51   }
52
53   @override
54   Widget build(BuildContext context) {
55
56     return Scaffold(
57       bottomNavigationBar: BottomNavigationBar(
58         currentIndex: _selectedIndex,
59         onTap: (index) {
60           setState(() {
61             _selectedIndex = index;
62           });
63         },
64         items: [
65           BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
66           BottomNavigationBarItem(icon: Icon(Icons.contact_page), label: "Contact"),
67           BottomNavigationBarItem(icon: Icon(Icons.settings), label: "Settings"),
68         ],
69       ), // BottomNavigationBar
70       body: _getSelectedPage(),
71     ); // Scaffold
72   }
73 }
```

เลือกเนื้อหาที่สร้างเตรียมไว้
มาแสดงในส่วนของ body
ตาม index ที่เลือก



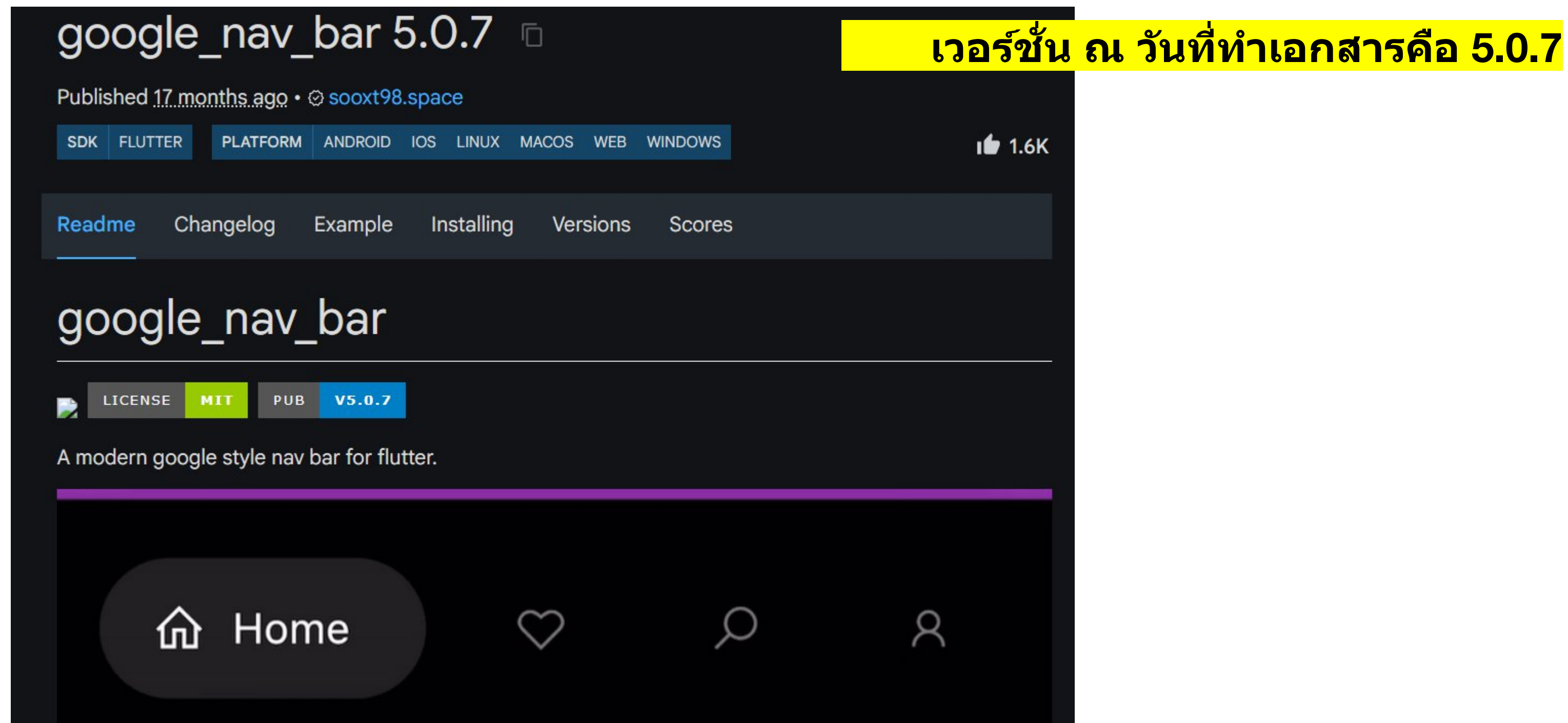
ส่วนของ BottomNavigationBar

ทดสอบการทำงาน **TABS** จากไฟล์ตัวอย่าง

- สร้างโปรเจกต์ใหม่ชื่อว่า *tabs_example*
- ดาวน์โหลดไฟล์ตัวอย่างจาก [ที่นี่](#)
- ให้แตก ZIP ไฟล์และนำ **/lib** วางทับลงในโปรเจกต์ที่สร้างใหม่
- ทดสอบการทำงานโดยการรันไฟล์ `main_xx` ภายใน `/lib`

Bottom Navigation Bar – Package

- แนะนำให้ศึกษาเพิ่มเติมการใช้ Bottom Navigation โดยใช้ Package จาก pub.dev ซึ่งจะมีความน่าใช้ สวยงาม และมีความสามารถต่างๆ เพิ่มเติมมากมาย
- ตัวอย่าง เช่น package ชื่อ [google_nav_bar](#)

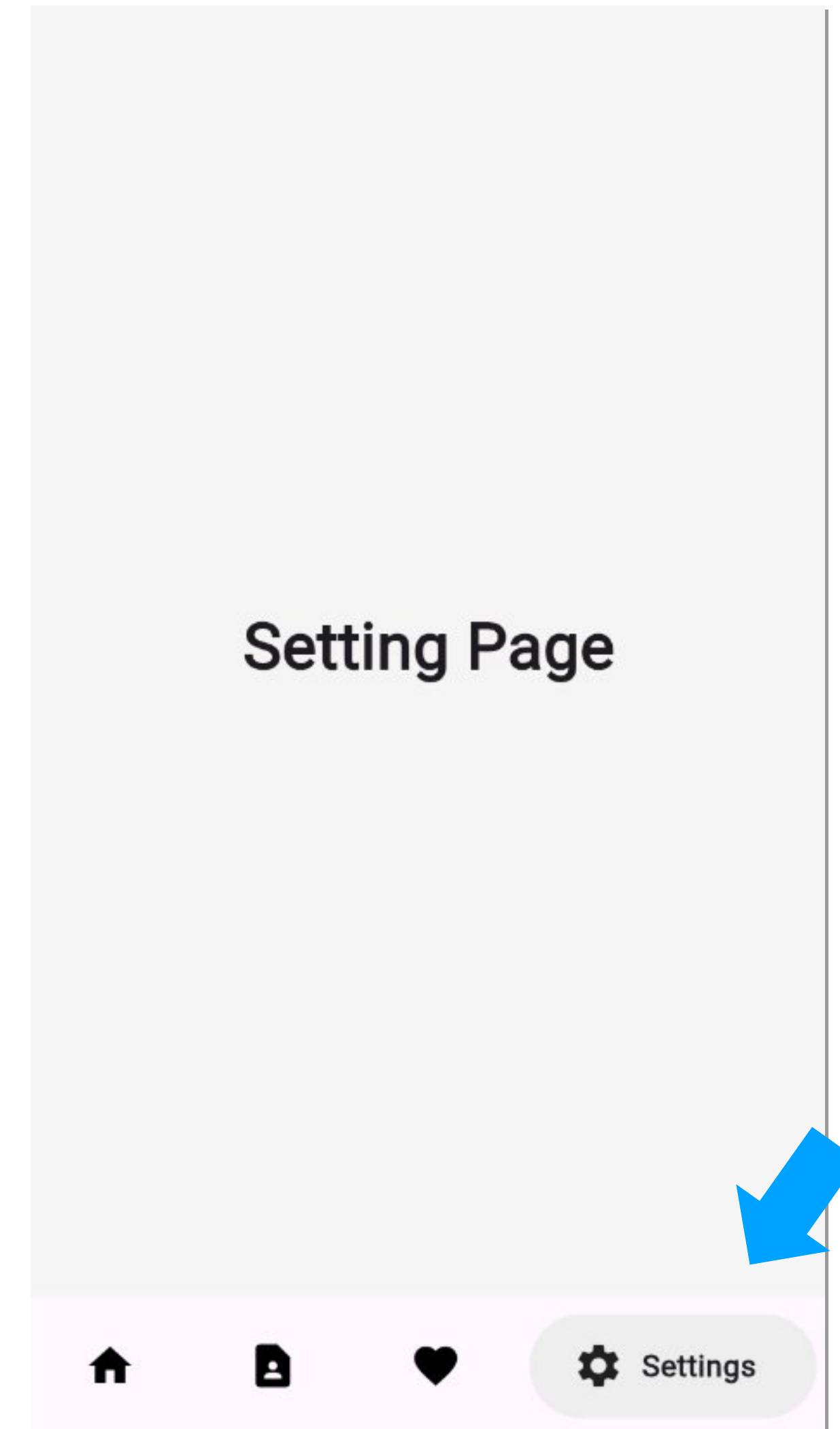


Bottom Navigation Bar – ตัวอย่าง

```
52 · @override
53 · Widget build(BuildContext context) {
54 ·   return Scaffold(
55 ·     bottomNavigationBar: Padding(
56 ·       padding: const EdgeInsets.all(8.0),
57 ·       child: GNav(
58 ·         gap: 8,
59 ·         padding: EdgeInsets.symmetric(horizontal: 20, vertical: 12),
60 ·         rippleColor: Colors.grey.shade300,
61 ·         tabBackgroundColor: Colors.grey.shade200,
62 ·         color: Colors.black,
63 ·         duration: Duration(milliseconds: 150),
64 ·         onTapChange: (index) {
65 ·           setState(() {
66 ·             _selectedIndex = index;
67 ·           });
68 ·         },
69 ·         selectedIndex: _selectedIndex,
70 ·         tabs: const [
71 ·           GButton(icon: Icons.home, text: 'Home'),
72 ·           GButton(icon: Icons.contact_page, text: 'Contact'),
73 ·           GButton(icon: Icons.favorite, text: 'Favorite'),
74 ·           GButton(icon: Icons.settings, text: 'Settings'),
75 ·         ],
76 ·       ), // GNav
77 ·     ), // Padding
78 ·     body: _getSelectedPage(),
79 ·   ); // Scaffold
80 · }
```

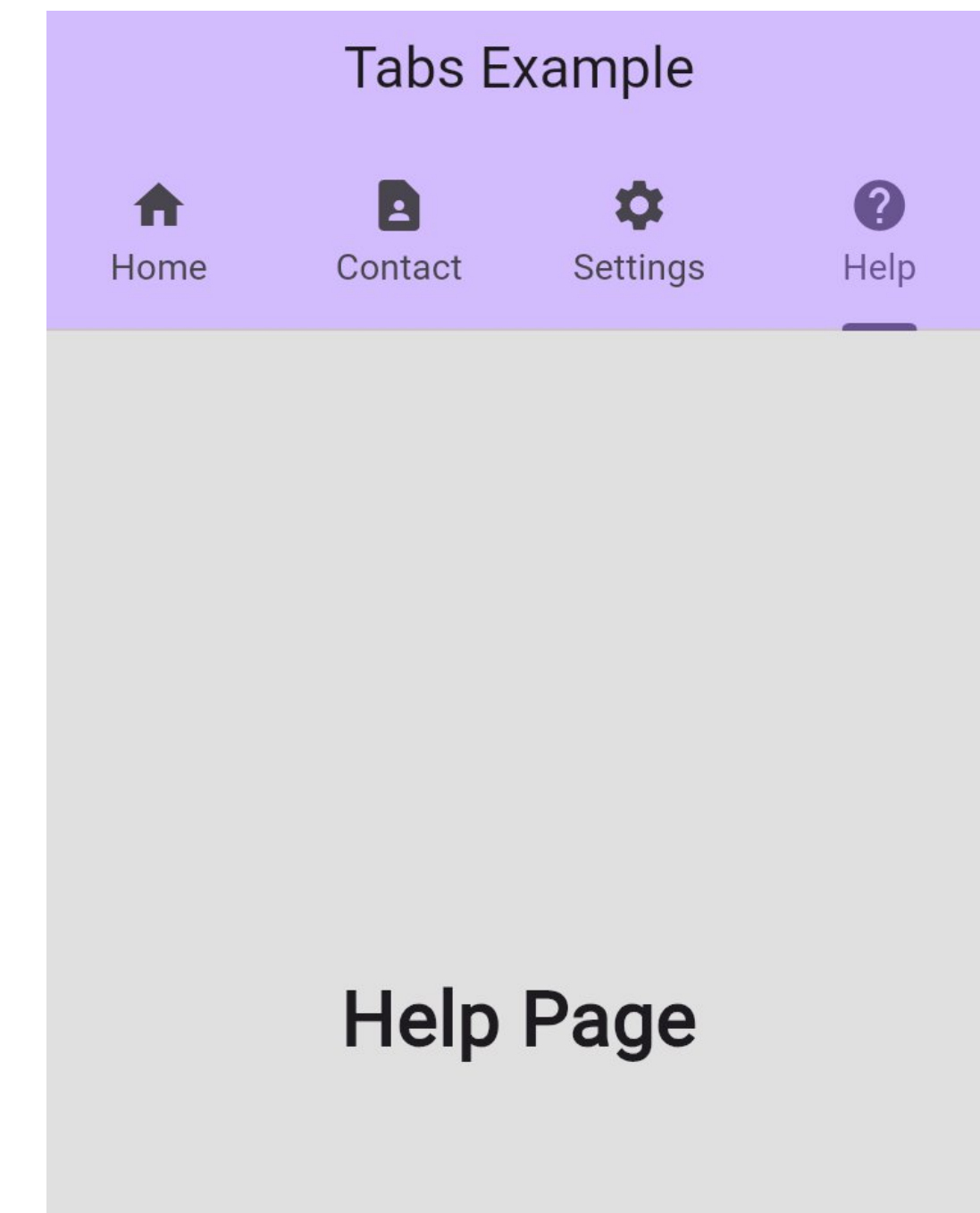
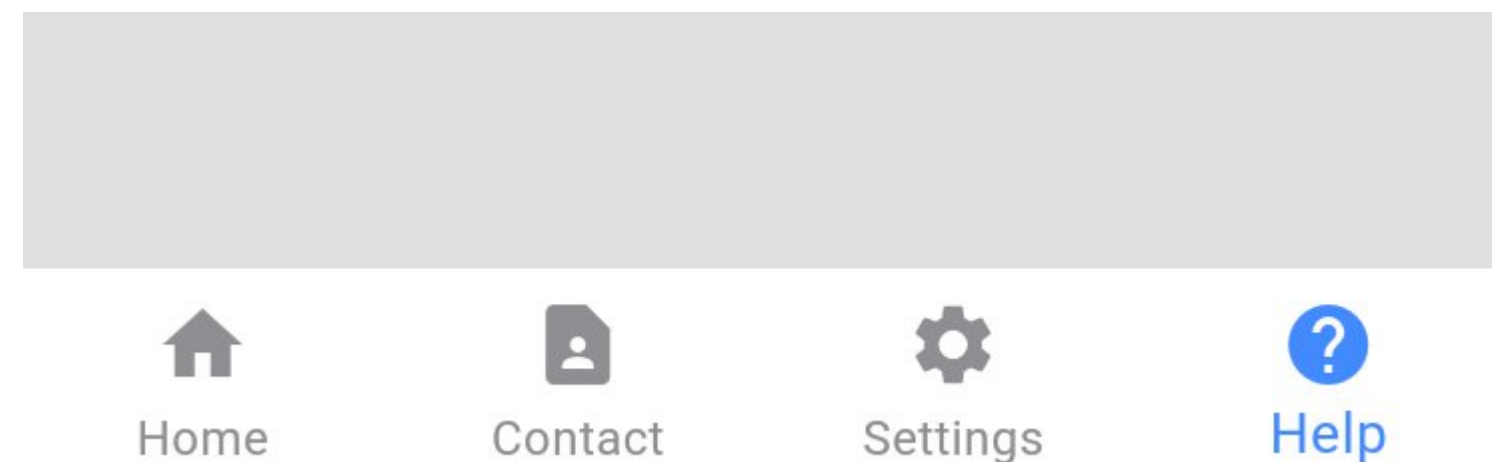
pubspec.yaml

```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the Cupertino
35   # icons to the CupertinoIcons
36   cupertino_icons: ^1.0.8
37   google_nav_bar: ^5.0.6
```



Challenge – เพื่อความเข้าใจ

1. จากตัวอย่าง TabBar ให้เพิ่ม Page จำนวน 1 Page เกี่ยวกับ Help โดยใช้ไอคอน Help และให้เนื้อหาภายใน Page แสดงข้อความว่า “Help Page”
2. จากตัวอย่างการใช้ BottomNavigationBar ให้เพิ่ม Help Page เข้าไปยังตัวเลือกด้วยเช่นเดียวกัน



ข้อสังเกต

- หากสีของ navigationBar กลมกลืนไปกับพื้นหลังจนมองไม่เห็น ให้กำหนดคุณลักษณะเพิ่มเติมของ BottomNavigationBar โดยระบุ backgroundColor, selectedItemColor และ unselectedItemColor ให้เป็นสีตามที่ต้องการ

Page Navigation – Basic

- สามารถอ่านรายละเอียดเพิ่มเติมได้ที่ <https://docs.flutter.dev/cookbook/navigation/navigation-basics>

- คำสั่งที่ใช้ในการเปลี่ยน Page

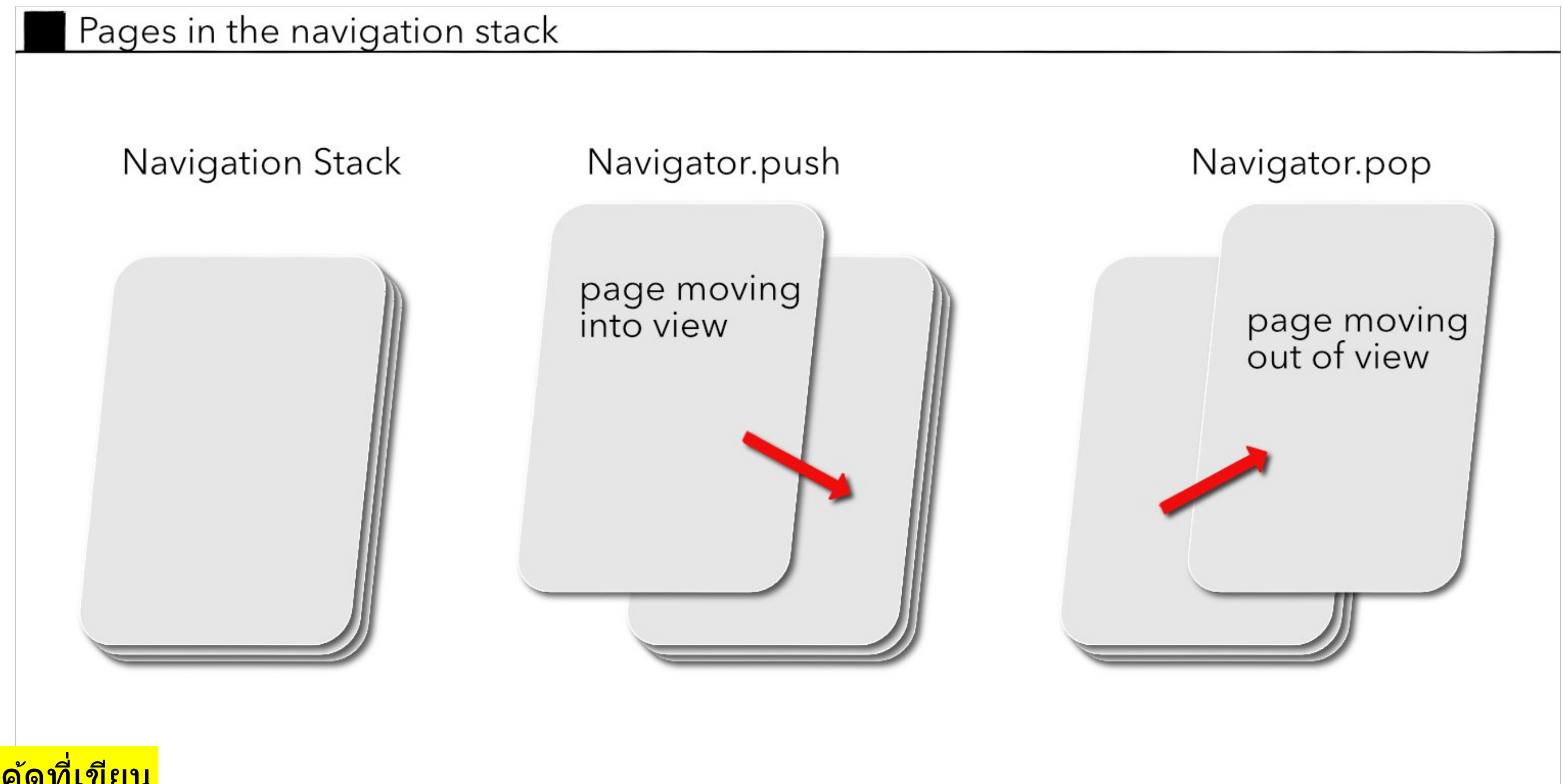
For Android

```
Navigator.push( context,  
  MaterialPageRoute(builder: (context) =>  
    const SecondRoute()),  
);
```

ชื่อ Screen ที่ต้องการนำมาแสดง ซึ่งจะเปลี่ยนแปลงไปตามโค้ดที่เขียน

- คำสั่งใช้ย้อนกลับ page ก่อนหน้า

```
Navigator.pop(context);
```



For IOS

```
Navigator.push( context,  
  CupertinoPageRoute(builder: (context) =>  
    const SecondRoute()),  
);
```


วิธีหลักในการทำ Navigation ใน Flutter

- Basic Navigation ใช้วิธีการดั้งเดิมคือ Push / Pop
 - `Navigator.push`: ใช้งานง่าย เหมาะกับ App ที่มีจำนวนหน้าไม่มาก และไม่ซับซ้อน โดยใช้ `MaterialPageRoute` เพื่อสลับหน้า Page A ไป Page B
 - `Navigator.pop`: ใช้สำหรับปิดหน้าปัจจุบันและกลับไปยังหน้าก่อนหน้า
- Named Routes (Named-based):
 - กำหนดชื่อเส้นทางใน MaterialApp โดยใช้ `routes` property เพื่อให้สามารถเรียกใช้ `Navigator.pushNamed(context, '/routeName')` ได้จากทุกที่ภายในแอป เหมาะสำหรับการจัดการ App ที่มี Page จำนวนมาก
- Router & RouterDelegate: เหมาะสำหรับแอปที่ต้องการ URL สำหรับเว็บ หรือ Deep Linking ที่ทำซับซ้อนในการทำ Navigation แนะนำให้ใช้แพ็คเกจช่วย เช่น `go_router`

วิธีหลักในการทำ Navigation ใน Flutter (cont.)

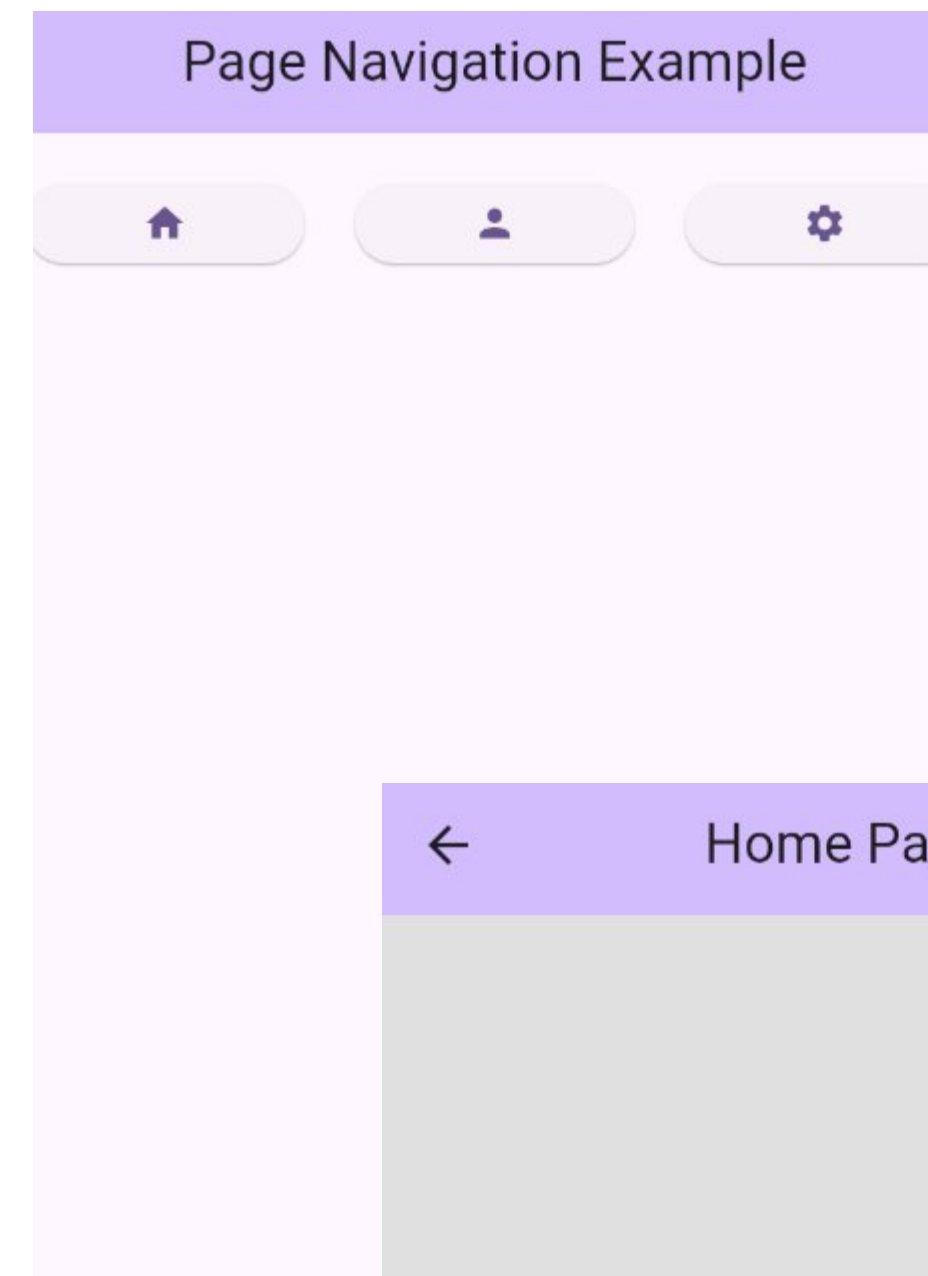
- การเปิดหน้าจอใหม่ด้วยการแทนที่หน้าจอเดิม โดยหน้าจอเก่าจะถูกลบออกจาก Stack ทันที
 - `Navigator.pushReplacement` : เมื่อผู้ใช้กดปุ่มย้อนกลับ (Back) จะไม่สามารถกลับมายังหน้าจอเดิมได้ เหมาะสำหรับหน้า Login ที่เมื่อเข้าสู่หน้าหลักแล้วไม่ต้องการให้ผู้ใช้กด Back กลับมาหน้า Login ได้อีก
 - `Navigator.pushAndRemoveUntil` : หน้าจอทั้งหมดที่อยู่ด้านล่างหน้าจอใหม่ จะถูกลบออกจนหมด (หรือตามเงื่อนไข) เหมาะสำหรับการออกจากระบบ (Logout) ที่ต้องการลบ Stack ของผู้ใช้ทั้งหมดแล้วกลับไปหน้าจอ Login หรือหน้าแรกของแอปพลิเคชัน

ทดสอบการทำงาน **Navigation** จากไฟล์ตัวอย่าง

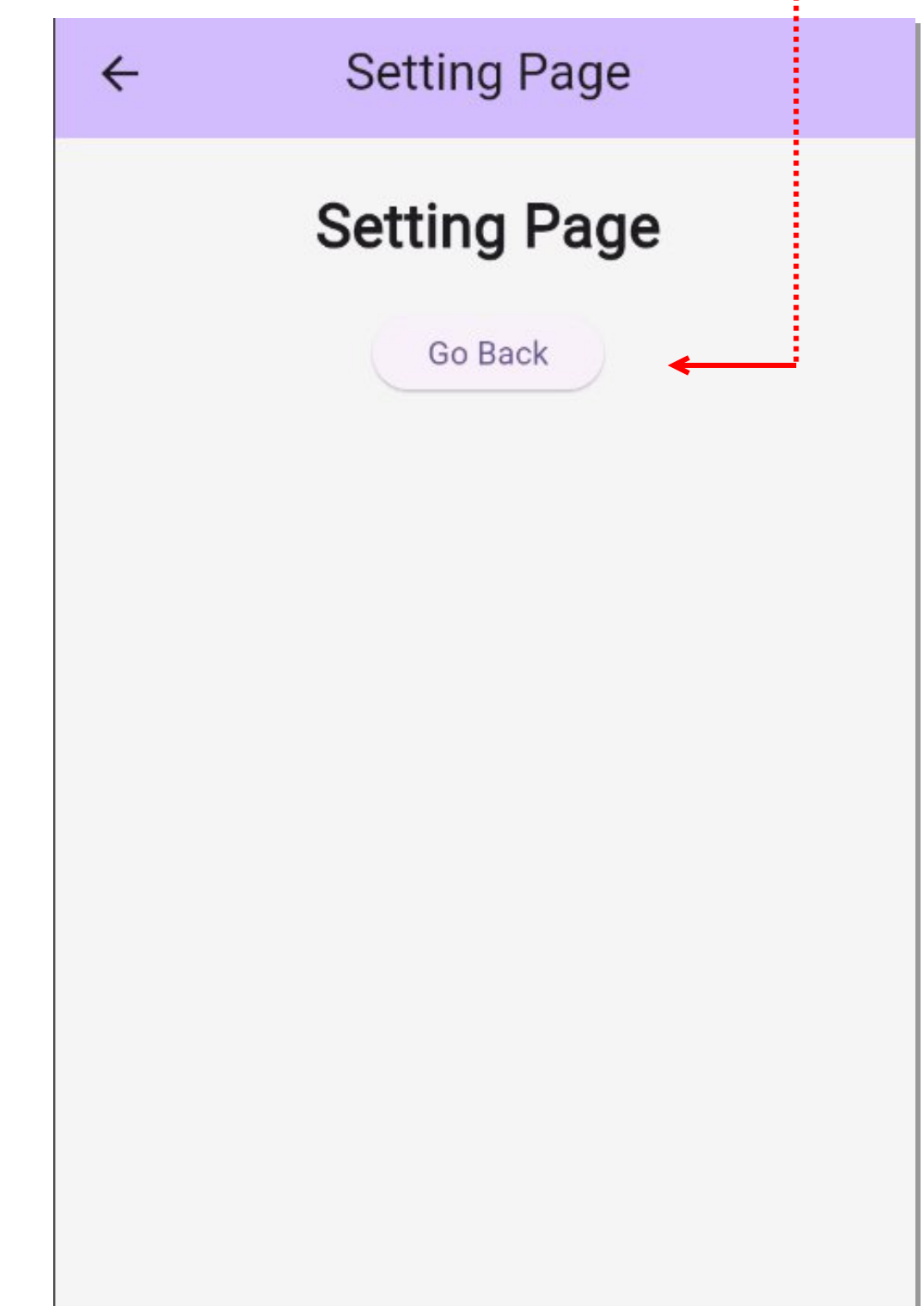
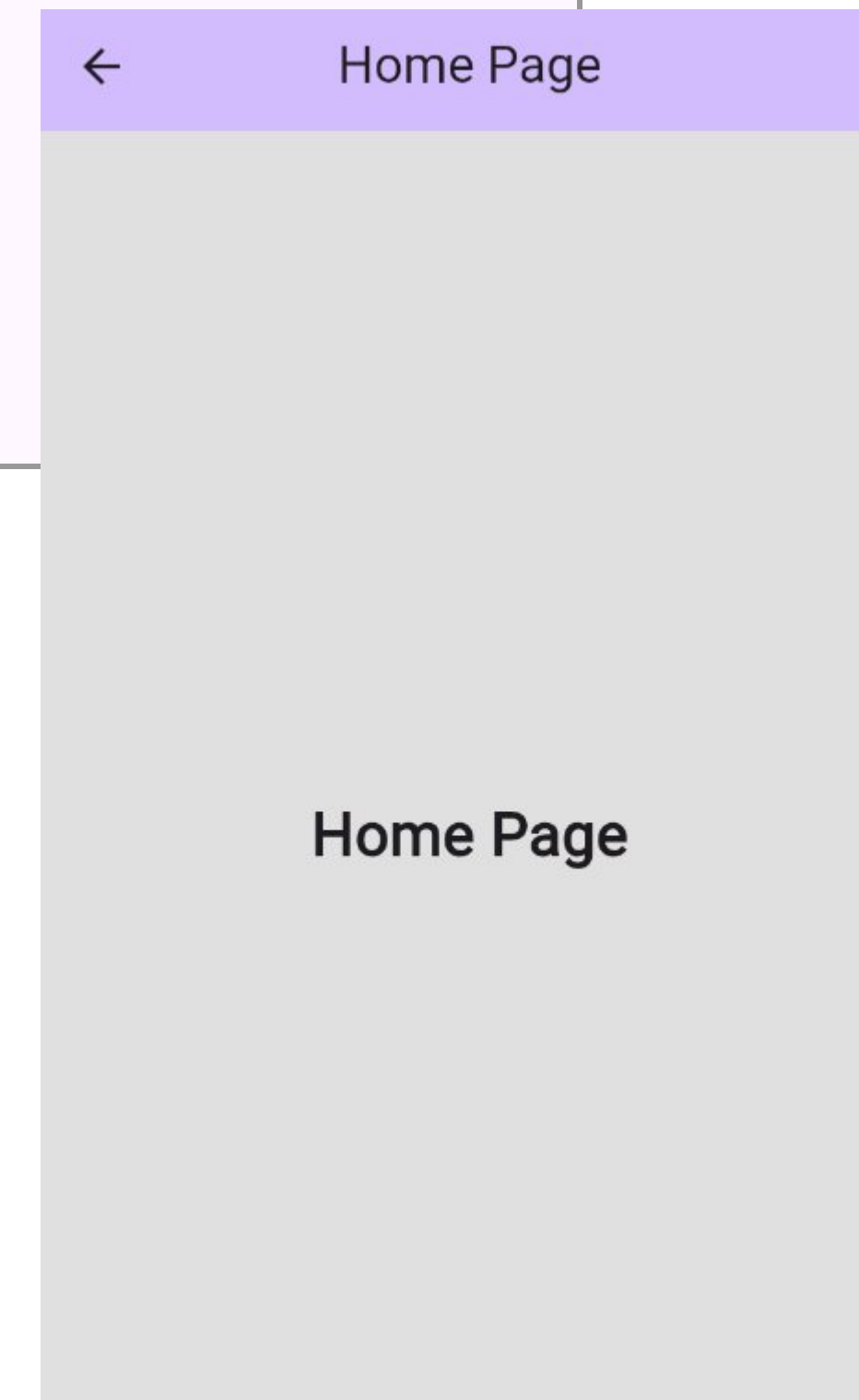
- สร้างโปรเจกต์ใหม่ชื่อว่า *navigate_example*
- ดาวน์โหลดไฟล์ตัวอย่างจาก [ที่นี่](#)
- ให้แตก ZIP ไฟล์และนำ **/lib** วางทับลงในโปรเจกต์ที่สร้างใหม่
- ทดสอบการทำงานโดยการรันไฟล์ `main_xx` ภายใน `/lib`

ตัวอย่างการทำงาน: push/pop

```
44 body: Column(  
45   children: [  
46     SizedBox(height: 20),  
47     Row(  
48       children: [  
49         Expanded(  
50           child: ElevatedButton(  
51             onPressed: () {  
52               Navigator.push(  
53                 context,  
54                 MaterialPageRoute(builder: (context) => HomePage()),  
55               );  
56             },  
57             child: Icon(Icons.home),  
58           ), // ElevatedButton  
59         ), // Expanded  
60         SizedBox(width: 20),  
61         Expanded(  
62           child: ElevatedButton(  
63             onPressed: () {  
64               Navigator.push(  
65                 context,  
66                 MaterialPageRoute(builder: (context) => ContactPage()),  
67               );  
68             },  
69             child: Icon(Icons.person),  
70           ), // ElevatedButton  
71         ), // Expanded  
72         SizedBox(width: 20),  
73         Expanded(  
74           child: ElevatedButton(  
75             onPressed: () {  
76               Navigator.push(  
77                 context,  
78                 MaterialPageRoute(builder: (context) => SettingPage()),  
79               );  
80             },  
81             child: Icon(Icons.settings),  
82           ), // ElevatedButton  
83         ), // Expanded  
84       ],  
85     ), // Row  
86   ],  
87 ), // Column
```



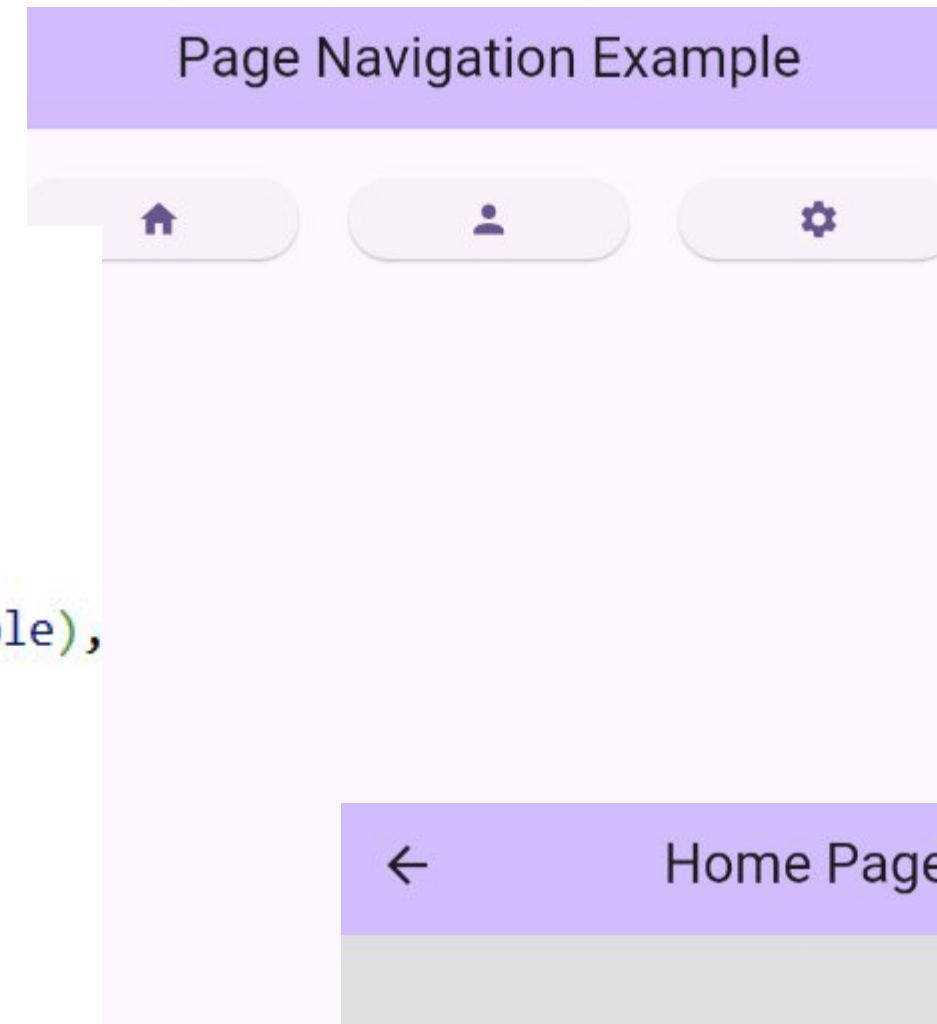
```
20 child: Column(  
21   children: [  
22     SizedBox(height: 20),  
23     Text(  
24       "Setting Page",  
25       style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold),  
26     ), // Text  
27     SizedBox(height: 20),  
28     ElevatedButton(  
29       onPressed: () {  
30         Navigator.pop(context);  
31       },  
32       child: Text("Go Back"),  
33     ), // ElevatedButton  
34   ],  
35 ), // Column
```



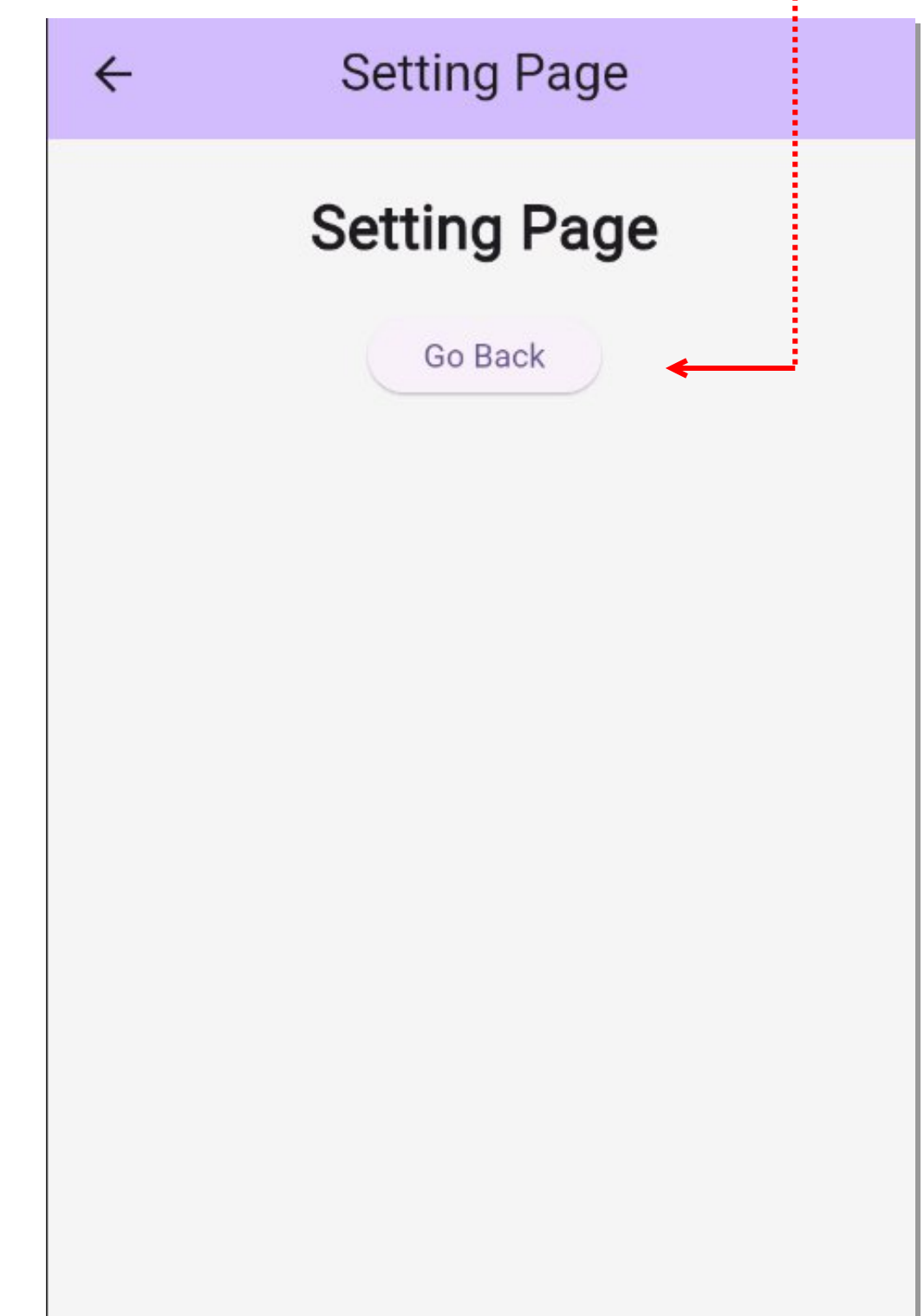
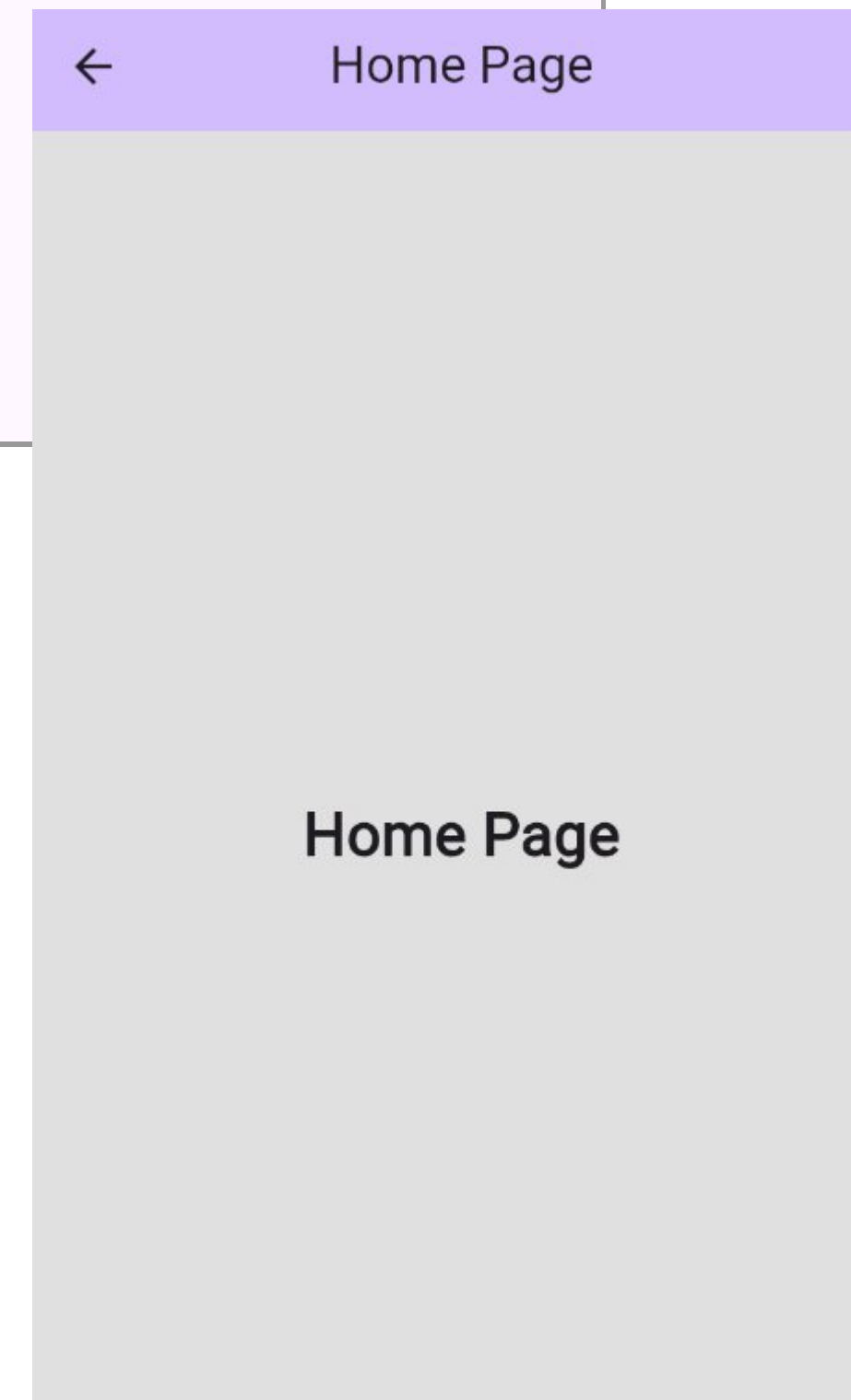
ตัวอย่างการทำงาน: Routes

```
15  @override
16  Widget build(BuildContext context) {
17    return MaterialApp(
18      title: 'Flutter Demo',
19      debugShowCheckedModeBanner: false,
20      theme: ThemeData(
21        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
22      ), // ThemeData
23      routes: {
24        '/home': (context) => HomePage(),
25        '/contact': (context) => ContactPage(),
26        '/setting': (context) => SettingPage(),
27      },
28      home: MyHomePage(),
29    ); // MaterialApp
30  }
```

```
51  body: Column(
52    children: [
53      SizedBox(height: 20),
54      Row(
55        children: [
56          Expanded(
57            child: ElevatedButton(
58              onPressed: () {
59                Navigator.pushNamed(context, '/home');
60              },
61              child: Icon(Icons.home),
62            ), // ElevatedButton
63          ), // Expanded
64          SizedBox(width: 20),
65          Expanded(
66            child: ElevatedButton(
67              onPressed: () {
68                Navigator.pushNamed(context, '/contact');
69              },
70              child: Icon(Icons.person),
71            ), // ElevatedButton
72          ), // Expanded
73          SizedBox(width: 20),
74          Expanded(
75            child: ElevatedButton(
76              onPressed: () {
77                Navigator.pushNamed(context, '/setting');
78              },
79              child: Icon(Icons.settings),
80            ), // ElevatedButton
81          ), // Expanded
82        ],
83      ), // Row
84    ],
85  ), // Column
```



```
20  child: Column(
21    children: [
22      SizedBox(height: 20),
23      Text(
24        "Setting Page",
25        style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold),
26      ), // Text
27      SizedBox(height: 20),
28      ElevatedButton(
29        onPressed: () {
30          Navigator.pop(context);
31        },
32        child: Text("Go Back"),
33      ), // ElevatedButton
34    ],
35  ), // Column
```



Notes

- การเปิด Page ใหม่โดยใช้ push/pop หรือ pushNamed ผ่าน Navigator สัญลักษณ์ปุ่ม ย้อนกลับ (<) จะปรากฏโดยอัตโนมัติก็ต่อเมื่อ Page ที่เปิดขึ้นมา นั้น มีการใส่ code เพื่อแสดง AppBar ด้วยเท่านั้น หากไม่มีส่วนนี้อยู่ใน Page ที่เปิดใหม่สัญลักษณ์นี้ จะไม่ปรากฏ
 - ส่วนของ AppBar เดิมในหน้าก่อน ก็จะถูกทับด้วยหน้าตาของ Page ใหม่ทั้งหมด
 - หากใช้ Bottom Navigation Bar เปิด Page ใหม่ด้วยวิธีใช้ Navigator ตัวเมนู Bar ด้านล่างก็จะถูกทับด้วยเช่นกัน หากต้องการให้ส่วนของเมนู Bar ด้านล่างยังคงอยู่ แนะนำให้หลีกเลี่ยงการใช้ Navigator ในการเปิด Page แต่ให้ใช้วิธีเปลี่ยน หรือแทนที่ส่วนของ body ในหน้าหลักให้เป็นเนื้อหาของ page ใหม่
- (หรือนอกจากนี้ยังสามารถเลือกใช้ package ชื่อ [persistent_bottom_nav_bar](#) ช่วยได้)

Challenge – เพื่อความเข้าใจ

- จากไฟล์ **help_page.dart** กอนหน้านี้ที่สร้างเพิ่มไว้ใน folder: **pages**
- ให้สร้างปุ่ม 1 ปุ่ม ภายในหน้าหลัก ใช้สำหรับกดเพื่อแสดงหน้า **HelpPage**
 - เลือกแก้ไขภายใน **main_pushpop.dart** หากใช้วิธี push/pop
 - หรือแก้ไขภายใน **main_route.dart** หากใช้วิธี pushNamed

Send/Receive data between pages

- เราสามารถส่งค่าใดๆ ไปพร้อมกับการ Navigate ไปยังหน้าใหม่ได้ และสามารถรับค่าใด ๆ ที่ return กลับมาได้ด้วยเช่นกัน

ส่งค่าไปยัง Page อื่น

```
Navigator.push( context, MaterialPageRoute(  
  builder: (context) => SecondScreen(  
    data: 'sending data',  
  ),  
), // MaterialPageRoute  
); // Navigator
```

Object หรือสิ่งที่ต้องการส่งไป

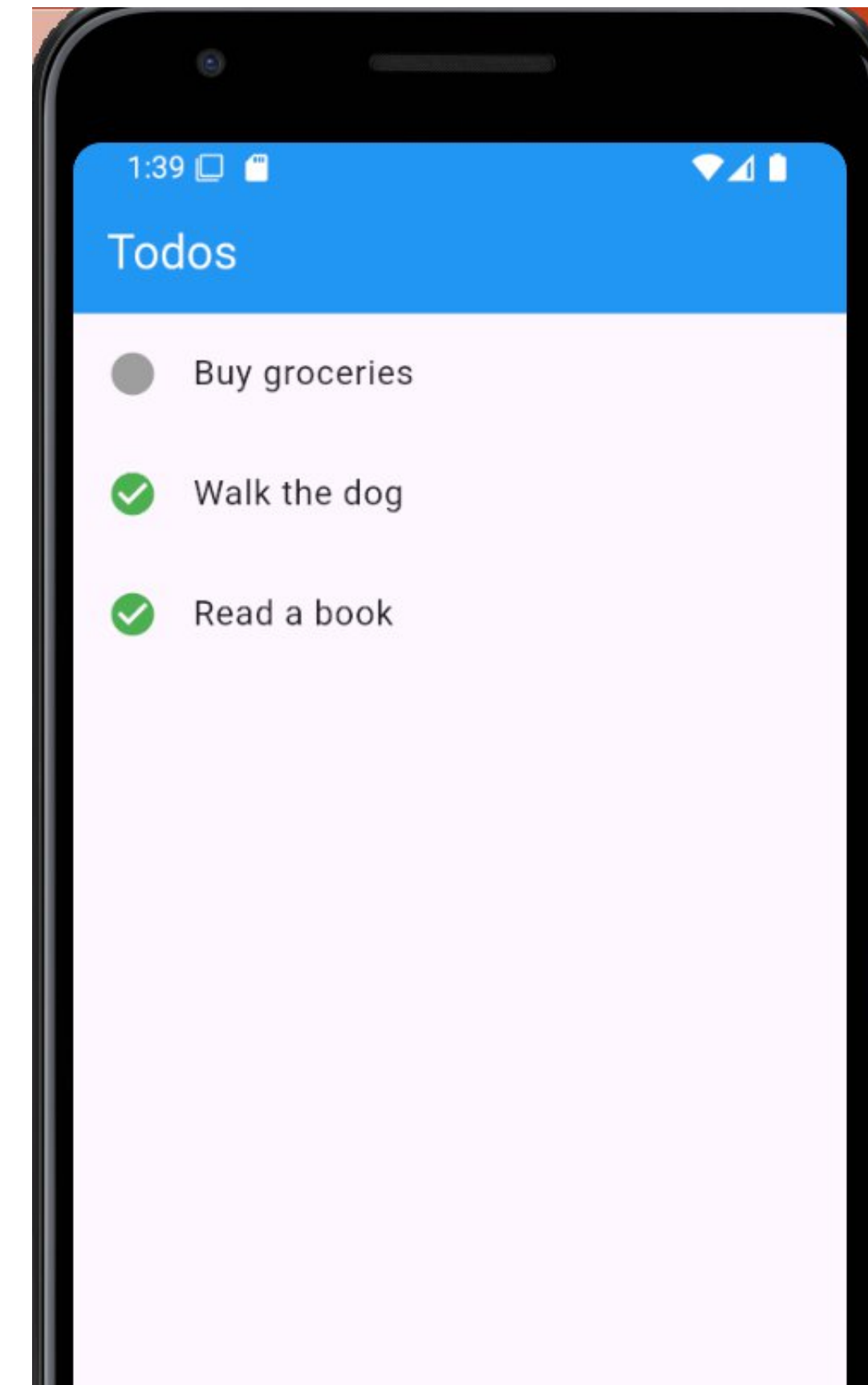
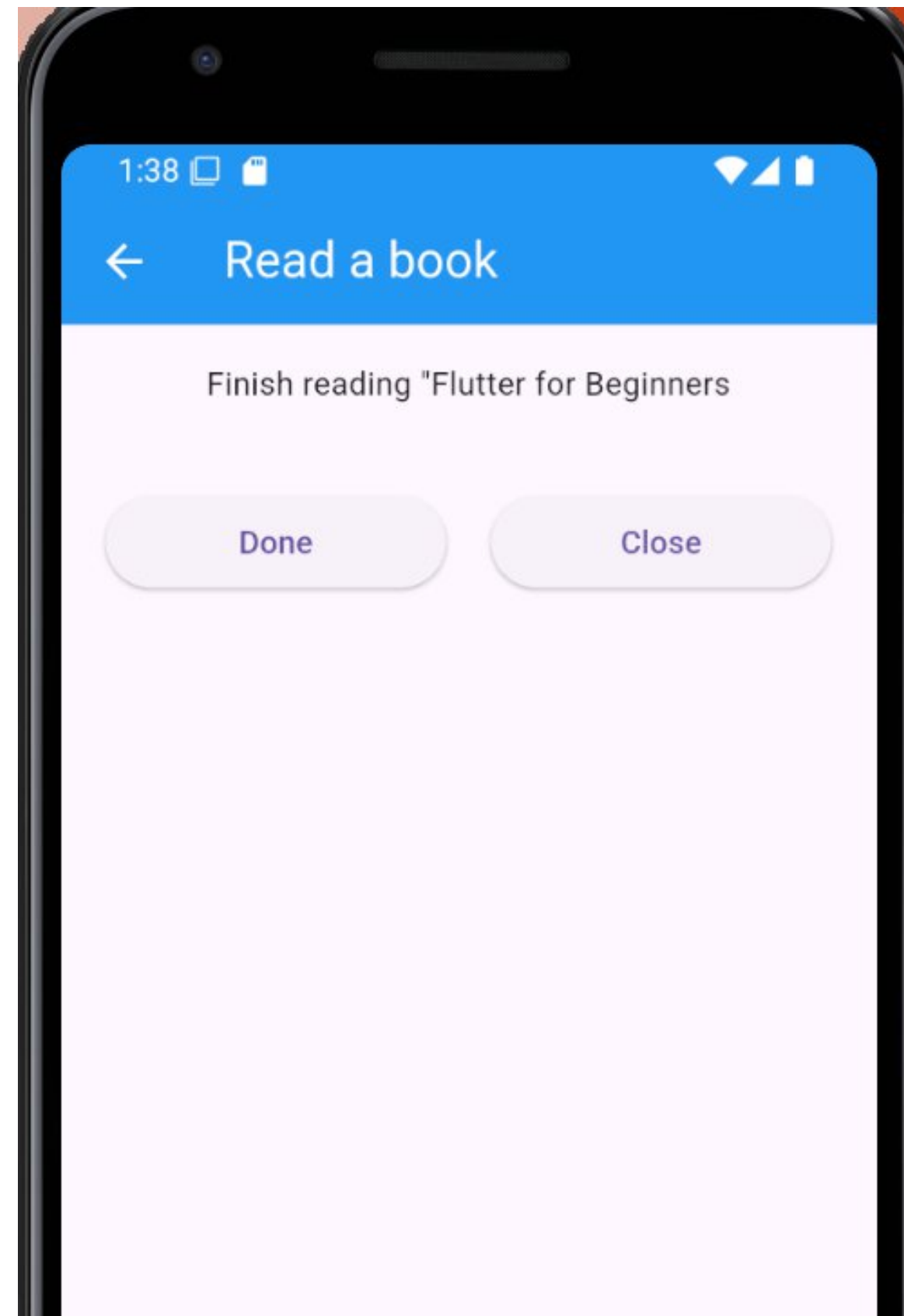
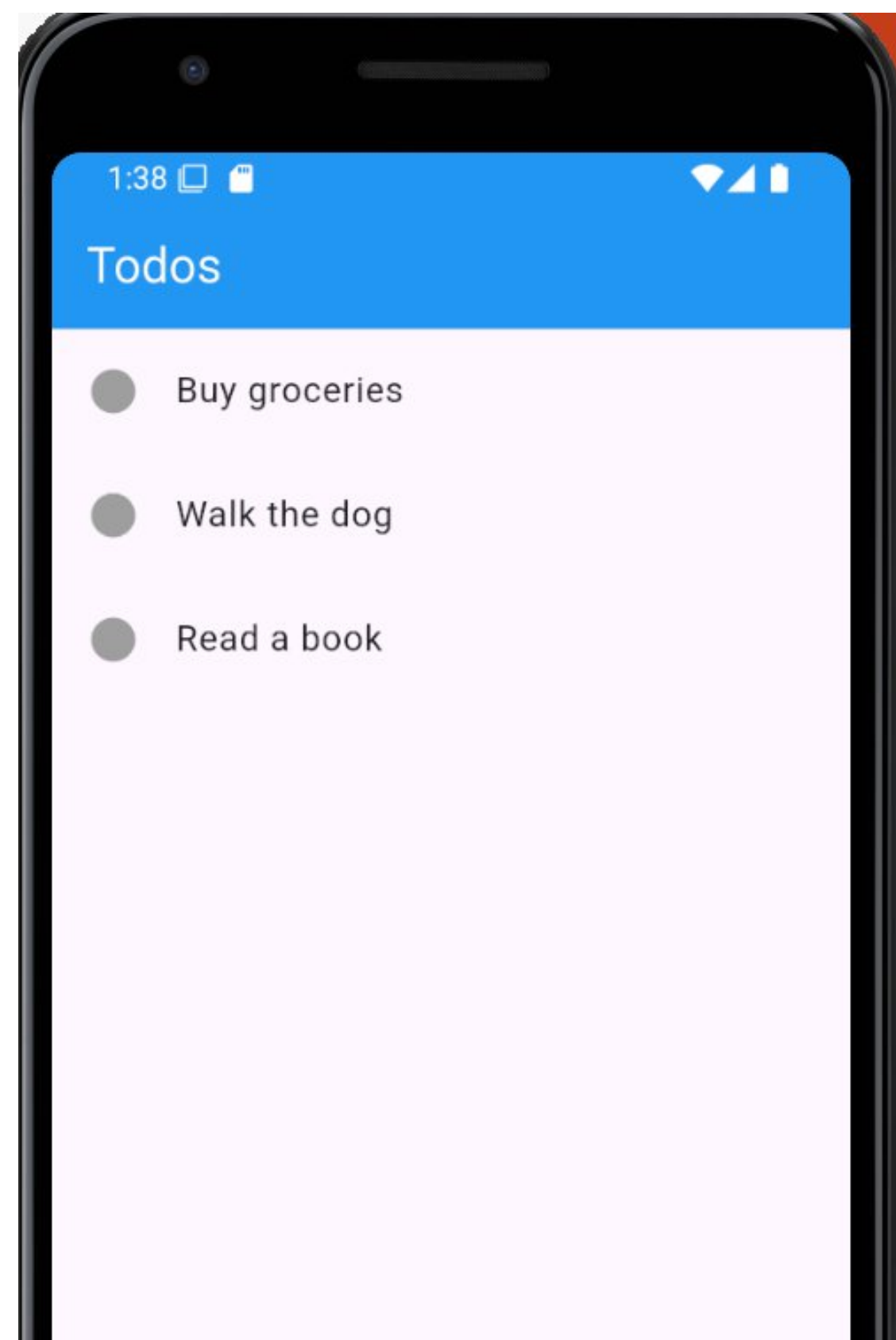
ชื่อตัวแปรของผู้รับที่กำหนดไว้ ซึ่งจะเปลี่ยนแปลงไปตามโปรแกรมที่เขียน ผู้ส่งจะต้องอ้างอิงให้ตรงกัน

การส่งค่ากลับไปยัง Page ที่เรียกมา

```
// Close the screen and return "Yes" as the result.  
Navigator.pop(context, 'Yes');
```

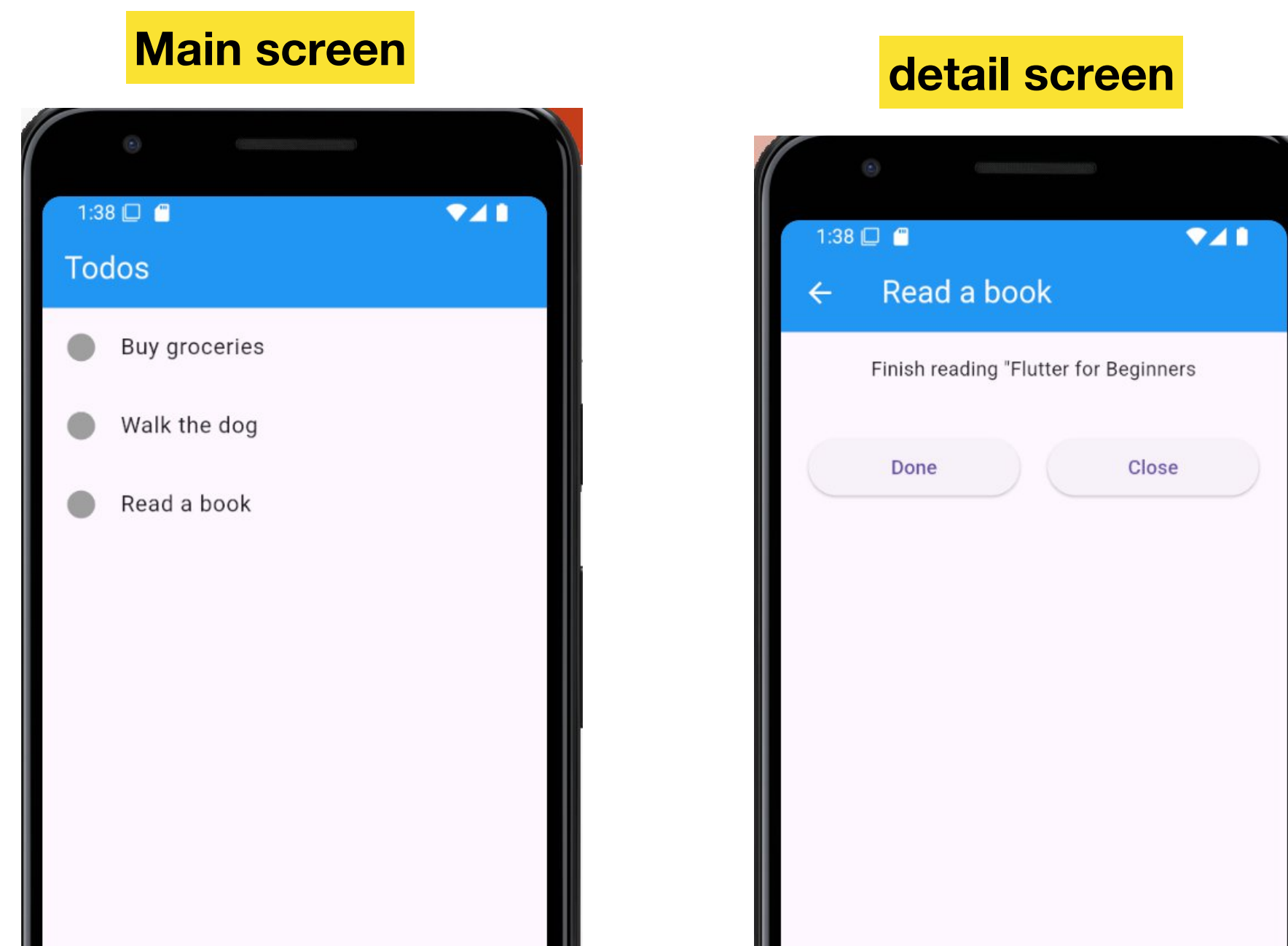
Object หรือสิ่งที่ต้องการส่งกลับ

ตัวอย่าง – การรับส่งข้อมูลระหว่าง Page



ตัวอย่าง – การรับส่งข้อมูลระหว่าง Page

- สร้าง Flutter Project ใหม่ชื่อว่า *navigate_withparam*
- สร้างไฟล์ภายใน lib จำนวน 2 ไฟล์ชื่อว่า *todo.dart* และ *detailscreen.dart*
- รายละเอียดโค้ดแสดงในหน้าถัดไป
- หรือ download ได้จาก [ที่นี่](#)



ตัวอย่าง – การรับส่งข้อมูลระหว่าง Page (cont.)

- โค้ดภายในไฟล์ **todo.dart**

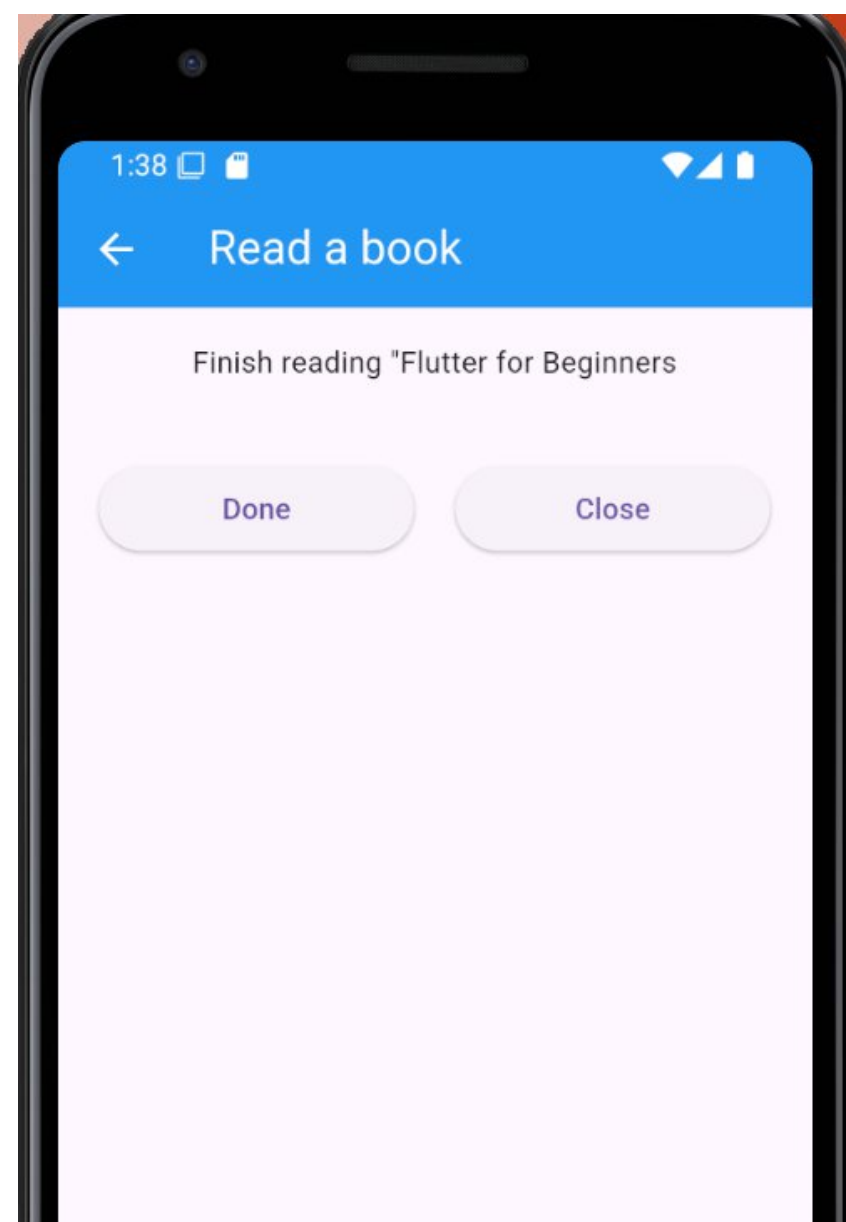
```
class Todo {  
  String title;  
  String description;  
  bool isCompleted = false;  
  
  Todo(this.title, this.description, this.isCompleted);  
}
```


ตัวอย่าง detailscreen.dart

```
import 'package:flutter/material.dart';
import 'todo.dart';

class DetailScreen extends StatelessWidget {
  // In the constructor, require a Todo.
  const DetailScreen({super.key, required this.todo});

  // Declare a field that holds the Todo.
  final Todo todo;
```



ต่อ

```
@override
Widget build(BuildContext context) {
  // Use the Todo to create the UI.
  return Scaffold(
    appBar: AppBar(title: Text(todo.title)),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: Text(todo.description),
        ),
        SizedBox(
          height: 20,
        ),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            SizedBox(
              width: 150,
              child: ElevatedButton(
                onPressed: () {
                  // Handle the button press, e.g., mark as completed
                  todo.isCompleted = !todo.isCompleted;
                  Navigator.pop(context, todo.isCompleted); // Go back to the previous screen
                },
                child: Text(todo.isCompleted ? 'Not Done' : 'Done'),
              ),
            ),
            SizedBox(width: 20),
            SizedBox(
              width: 150,
              child: ElevatedButton(
                onPressed: () {
                  Navigator.pop(context, todo.isCompleted); // Go back to the previous screen
                },
                child: const Text('Close'),
              ),
            ),
          ],
        ),
      ],
    ),
  );
}
```

ตัวอย่าง main.dart

```
import 'package:flutter/material.dart';
import 'DetailScreen.dart';
import 'todo.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  // Using a static list of todos for demonstration.
  List<Todo> todos = [
    Todo('Buy groceries', 'Milk, Bread, Eggs', false),
    Todo('Walk the dog', 'Take the dog for a walk in the park', false),
    Todo('Read a book', 'Finish reading "Flutter for Beginners"', false),
  ];

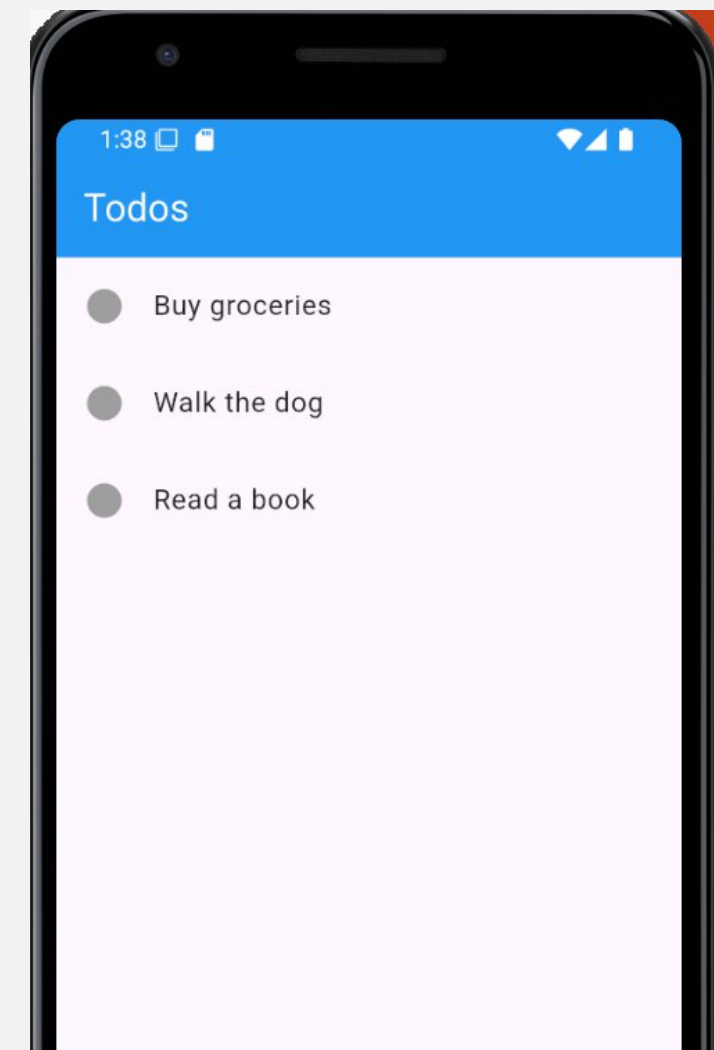
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Todo App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
        ),
      ),
      home: TodosScreen(todos: todos),
      // Passing list of todos to TodosScreen
    );
  }
}
```

ต่อ

```
class TodosScreen extends StatefulWidget {
  // Requiring the list of todos.
  const TodosScreen({super.key, required this.todos});
  final List<Todo> todos;

  @override
  State<TodosScreen> createState() => _TodosScreenState();
}

class _TodosScreenState extends State<TodosScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Todos')),
      //passing in the ListView.builder
      body: ListView.builder(
        itemCount: widget.todos.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(widget.todos[index].title),
            leading:
              widget.todos[index].isCompleted
                ? Icon(Icons.check_circle, color: Colors.green)
                : Icon(Icons.circle, color: Colors.grey),
            onTap: () async {
              final result = await Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => DetailScreen(todo: widget.todos[index]),
                ),
              );
              setState(() {
                if (result == null) return; // If no result, do nothing
                widget.todos[index].isCompleted = result;
              }); // Refresh the UI
            },
          );
        },
      ),
    );
  }
}
```



Q/A

การบ้าน #3

- ให้สร้าง App สำหรับแสดงเมนูของร้านอาหารชื่อ **ICT-Cafe** โดยมีข้อกำหนดดังนี้
 - ให้เลือกใช้ TabBar หรือ Bottom Navigation Bar อย่างใดอย่างหนึ่งในการเลือกเพื่อแสดงรายการอาหารตามกลุ่มต่าง ๆ
 - กลุ่มของอาหารประกอบด้วย 3 กลุ่ม ได้แก่ อาหารคาว อาหารหวาน และเครื่องดื่ม
 - ในแต่ละกลุ่มให้มีอาหารอย่างน้อย 6 รายการขึ้นไป โดยเนื้อหาจะต้องมี รูปของอาหาร ชื่อของอาหาร และราคา
 - และมี 1 ช่องทางที่เมื่อเลือกแล้ว ให้เปิดหน้าต่างขึ้นมาพร้อมกับข้อมูล รูป logo ร้าน ชื่อร้าน ที่อยู่ เวลาเปิด/ปิด และช่องทางการติดต่อ (ให้ใช้ข้อมูลสมมติทั้งหมด)
- ส่งภายในคาบเรียนครั้งถัดไป (ให้ใช้ Chrome ในการแสดงผลได้)